2factor

Perpetual senior / junior tranche protocol

Smart Contract Security Assessment

Repository	`github.com/fragmentsorg/2factor`
Commit	`2e527dac9890dd8a857edeae57285d09fdef4375`
Branch	`audit-update` (tree matches `HEAD`)
Audit date	28 August 2026
Report version	1.0
Classification	Confidential — recipient internal use
Auditor	Independent model-assisted review (Claude Opus 5, 1M context, operated via Claude Code)

Engagement metadata

Audit period	`2026-08-28T14:01:31Z` — `2026-08-28T15:38:37Z`
Report generated	`2026-08-28T15:38:37Z`
Lead model	`claude-opus-5[1m]` (Claude Opus 5, 1M context)
Reasoning / effort setting	Not exposed by runtime
Repository commit	`2e527dac9890dd8a857edeae57285d09fdef4375`
Report version	1.0
Time zone	All timestamps UTC (ISO 8601, seconds precision)

This assessment is independent work product. It is not affiliated with, endorsed by, or derived from the templates or methodology of any other security firm.

Contents

1 Document control **1**
- 1.1 Version history . . . 1
- 1.2 Engagement . . . 1
- 1.3 Scope . . . 1
- 1.4 Execution provenance . . . 1
- 1.5 Disclaimer . . . 2

2 Executive summary **4**
- 2.1 What the protocol does . . . 4
- 2.2 What was reviewed . . . 4
- 2.3 Overall posture . . . 4
- 2.4 Direct answers . . . 5
- 2.5 Findings by severity . . . 6

3 Scope and methodology **7**
- 3.1 In-scope files . . . 7
- 3.2 Out of scope . . . 7
- 3.3 Environment and reproduction . . . 7
- 3.4 Method . . . 7
- 3.5 Adversarial strategies applied . . . 8
- 3.6 Material limitations . . . 8

4 Architecture and trust model **10**
- 4.1 System overview . . . 10
- 4.2 Asset flow and custody . . . 10
- 4.3 Roles and privileges . . . 12
- 4.4 Trust boundaries . . . 12
- 4.5 Upgrade model . . . 14
- 4.6 Oracle model . . . 14
- 4.7 State machines . . . 14
- 4.8 Off-chain assumptions and liveness dependencies . . . 14

5 Security properties tested **15**

6 Findings summary **17**

7 Detailed findings **19**
- 7.1 MED-01 — The fee curve is a hard gate, and it closes on redemptions . . . 19
- 7.2 MED-02 — Perpetual notes have no first-depositor protection . . . 22
- 7.3 MED-03 — One price feed gates every entry point, exits included . . . 24
- 7.4 MED-04 — The AUM fee is unbounded and charged per call, not per unit time . . . 26
- 7.5 LOW-01 — `vmDR` is not clipped, and the fee curve overflows on it . . . 28
- 7.6 LOW-02 — `deposit` takes collateral and mints nothing below one unit of value . . . 30
- 7.7 LOW-03 — `redeemImmature` burns senior without the paired junior . . . 31
- 7.8 LOW-04 — The controller trusts the fee policy's protocol-share ordering . . . 32
- 7.9 LOW-05 — `upkeep()` reverts `NoOp`, so the routers' upkeep flags are unusable . . . 33
- 7.10 LOW-06 — The vault charges depositors for resting underlying but never pays redeemers for it . . . 34
- 7.11 LOW-07 — `updateTargetDR` accepts a zero lower bound . . . 36
- 7.12 LOW-08 — Stateless routers grant unbounded approvals to caller-supplied addresses 37

7.13 LOW-09 — `mature()` is permissionless with no deadline, so the caller picks the settlement price . . . 38
7.14 LOW-10 — `updateBondConfig` under-validates and can stop issuance . . . 40
7.15 LOW-11 — Rollover divides by `valueRatio` with no zero guard . . . 41
8 Informational observations **42**
9 Economic and design risks **45**
9.1 The rebalance is a scheduled, predictable value transfer . . . 45
9.2 MEV on the vault rebalance is bounded by `oracleDeviationPerc` . . . 45
9.3 The senior leg's protection is only as good as the settlement price . . . 45
9.4 Junior wipeout is a discontinuity, not a slope . . . 45
9.5 Dependency on external venues . . . 46
9.6 Bad debt is not modelled . . . 46
10 Centralization and privileged operations **47**
11 Testing results **49**
11.1 Existing suites . . . 49
11.2 Compiler warnings . . . 49
11.3 Added tests . . . 49
11.4 Coverage . . . 49
11.5 Static analysis . . . 50
11.6 Analyses that were attempted and did not complete . . . 51
11.7 Meaningful negative results . . . 51
12 Conclusion **53**
12.1 Overall assessment . . . 53
12.2 Confirmed severe risks . . . 53
12.3 Unresolved issues . . . 53
12.4 Assumptions this report rests on . . . 53
12.5 Recommended actions before deployment . . . 53
12.6 Residual risk . . . 54
A File inventory and review status **55**
B Privileged function inventory **57**
C Tool versions and commands **60**
D Candidate-finding disposition **62**
E Prior audit verification **65**
F Glossary **66**
G Disclaimer **68**

1 Document control

1.1 Version history

Version	Date (UTC)	Description
1.0	2026-08-28	Initial release. Full-scope review of `core-contracts` and `vaults` at commit `2e527da`.

1.2 Engagement

Protocol	2factor (`2factor.finance`), formerly Fragments
Repository	`github.com/fragmentsorg/2factor` (monorepo, yarn 4 workspaces)
Commit reviewed	`2e527dac9890dd8a857edeae57285d09fdef4375`
Working tree	Local checkout at `/Users/nithin/projects/ampl-core/2ff`, branch `audit-update`. Uncommitted changes were confined to repository plumbing (asset/script relocation, README, CI workflow, `audits/` move) and did not touch any Solidity source.
Audit period	`2026-08-28T14:01:31Z` — `2026-08-28T15:38:37Z`
Report status	Final, version 1.0
Compiler	solc `0.8.35+commit.47b9dedd`, EVM target `cancun`, optimizer on (runs 800), `viaIR` enabled
Build system	Hardhat 2.29.0 (mocha + chai + ethers v6 + OpenZeppelin upgrades plugin)
Target chains	Not fixed in-repo. Deployment scripts target Ethereum mainnet and Sepolia; the vault DEX strategies are written against Base-style Curve and Uniswap v4 deployments, and `L2CLOracleWrapper` exists specifically for L2s. Chain selection materially changes several findings and is flagged where it does.
Prior review	PeckShield, report 2026-021, 16 March 2026, at commit `dc1e08e` with fixes at `0ff7ffb`. Six findings (1 Medium, 5 Low), all recorded as resolved or mitigated. Verified as remediated at the reviewed commit; see Appendix E.

1.3 Scope

In scope: every Solidity file under `core-contracts/contracts/` and `vaults/contracts/` excluding the `_test/` subtrees: 58 files, 11,001 physical lines. The complete per-file inventory with review status is Appendix A.

Out of scope, reviewed only as context and explicitly *not* assessed: the Go keeper under `ops/`, the subgraph under `subgraph/`, the research papers under `research/`, Hardhat Ignition deployment modules, Hardhat tasks, and all third-party dependencies (OpenZeppelin 5.6.1, Uniswap v4-core 1.0.2, Chainlink aggregators, Curve pools, CoW Protocol, Morpho Blue), which are treated as trusted.

1.4 Execution provenance

Every substantive finding in this report was produced by a single lead agent. No auxiliary review agents, no multi-agent orchestration, and no third-party scanning service were used. Automated

tooling (solhint, solidity-coverage, Slither) was run by that same agent, and its output is reported as tool output rather than as independent review.

Exec ID	Agent role	Exact model	Effort	Started (UTC)	Purpose
EX-01	Lead auditor	`claude-opus-5[1m]`	Not exposed	`2026-08-28T14:01:31Z`	Scoping, system reconstruction, manual review, threat model, proof-of-concept construction, report authoring
EX-02	Tooling (same agent)	—	—	`2026-08-28T14:35:00Z`	solidity-coverage, solhint, Slither, compiler-warning capture
EX-03	Report build (same agent)	—	—	`2026-08-28T14:46:37Z`	LaTeX typesetting via tectonic, PDF render and page-by-page visual inspection

Total wall-clock audit duration	`1h 37m`
Active execution duration	Not reliably measurable. The runtime exposes no per-request CPU or token accounting to the agent, so no active-time figure is asserted.
Testing / fuzzing duration	`56m` of measured test and coverage execution (existing suites, added proof-of-concept suites, two coverage runs). No new long-running fuzz campaign was executed — see §3.6.
Report-generation duration	`52m`
Agent runtime version	Not exposed by runtime. The agent has no accessor for the Claude Code CLI version string from inside the session.
Operating environment	macOS 15.7.7, x86_64, Node.js v22.15.0, Hardhat EDR in-process network

Disclosure on model-assisted auditing. This assessment was produced by a large language model operating with source access, a local build, and a test runner. Model-assisted review does not guarantee the absence of vulnerabilities. It is subject to the same limits as any single-reviewer engagement — bounded attention, bounded time, and reliance on the reviewer's own hypotheses about where defects live, and to failure modes specific to the method, including confident-sounding reasoning about code paths that were never executed. Every claim in the Detailed Findings section is backed either by an executable proof-of-concept committed with this report or by an explicit source citation; claims that rest on reasoning alone are labelled as such and given a correspondingly lower confidence rating. This report is not a warranty, an insurance product, or investment advice.

1.5 Disclaimer

This review covers the specific commit named above. It does not cover deployment configuration, key management, off-chain infrastructure, front ends, economic parameter choices made after this date, or any code not present at that commit. A clean test run is not evidence of security, and high line coverage is not evidence of adversarial coverage. The absence of a Critical or High finding

in this report means only that none was identified under the assumptions and within the time recorded here.

2 Executive summary

2.1 What the protocol does

2factor splits an underlying asset's volatility into two perpetual ERC-20 claims. A *bond* takes a deposit of the underlying and mints a senior and a junior tranche token against it in a fixed ratio, recording the oracle price at creation. At maturity the collateral is split: the senior gets a priority claim sized at its ratio scaled by the open-to-close price move, and the junior takes whatever is left. Two perpetual notes wrap rolling baskets of those tranches. **PerpSr** holds senior legs and **PerpJr** holds junior legs plus idle underlying. A controller rolls maturing tranches into fresh ones, moves value between the two notes to hold a target ratio, and prices every user operation through a fee policy that keys off how far the system sits from its targets. A vault layer on top holds those notes for depositors and rebalances them against a DEX, a whitelisted market maker, or a CoW Protocol solver.

The economic claim is that junior holders get moderate, non-liquidating leverage financed by senior holders who are paid a yield for absorbing it, and that sourcing leverage this way is cheaper than hedging it.

2.2 What was reviewed

All 58 non-test Solidity files across the two in-scope packages (11,001 lines) were read line by line, and every value-moving external entry point was traced end to end through the contracts it touches. The build was reproduced locally, the existing 2,476 tests were executed, coverage was re-measured, and twenty-three new proof-of-concept tests were written to confirm or refute specific hypotheses. Findings backed by a proof-of-concept are marked *Confirmed*; nothing is reported as exploitable on reasoning alone.

2.3 Overall posture

The codebase is unusually disciplined for its size. Accounting is centralised, rounding directions are deliberate and mostly documented, reentrancy guards are present on every state-mutating entry point, external calls are ordered after accounting in the paths that matter, and the mint/redeem share arithmetic preserves price-per-share exactly rather than approximately, which this review verified symbolically and by test. The prior PeckShield findings are genuinely fixed rather than papered over. Test coverage is real: 100% of statements and functions in both packages, with branch coverage at 93.9% and 96.8%.

No Critical or High finding was identified. No path was found by which an unprivileged actor can steal principal, mint unbacked notes, or drive the system insolvent under the tested assumptions.

The risks that do exist cluster in three places, and all three are about *availability and configuration* rather than theft:

1. **Redemption can be switched off by policy, in the states where holders most want out.** The op-fee curve is not only a price; it is a gate. `PerpController` reverts `OpDisabled` the moment a computed fee reaches 100%, and the fee curves shipped in `FeePolicy.init()` reach 100% at ordinary deviation levels. At a subscription deviation ratio of 2.0 or above a PerpSr holder cannot redeem *any* amount; at 0.66 or below a PerpJr holder cannot. Because the USD vault exits its position by flash-redeeming PerpSr, its depositors inherit the freeze. This is deliberate design, but it is a hard freeze with no override short of the owner replacing the fee policy (MED-01).

2. **A single Chainlink feed gates every entry point, exits included.** Tranche valuation reads the live price for any unmatured bond, and every controller function — including `redeem2` and `flashRedeem2` — starts by reading reserve state. One stale answer, one zero answer, or one L2 sequencer restart reverts the entire protocol, with no fallback feed and no degraded withdrawal path (MED-03).

3. **Two unbounded owner parameters.** The AUM fee accepts any value below 100%, and is charged per `rebalance()` call rather than per unit time; at 99% a single permissionless `rebalance()` mints 99× the outstanding supply to the fee collector and dilutes every holder to 1% (MED-04). Separately, the perpetual notes have no first-depositor protection: a plain ERC-20 transfer of a registered reserve tranche inflates price-per-share, and at low supply the next depositor mints zero notes and forfeits their deposit (MED-02). The vault layer defends exactly this with a locked `MINIMUM_LIQUIDITY`; the notes underneath it do not.

2.4 Direct answers

Were any direct fund-loss paths confirmed? None reachable by an unprivileged actor against a seeded, correctly configured system. One confirmed loss path (MED-02) requires the note supply to be at or near a single wei, which is a bootstrap condition. One confirmed dilution path (MED-04) requires a malicious or compromised owner.

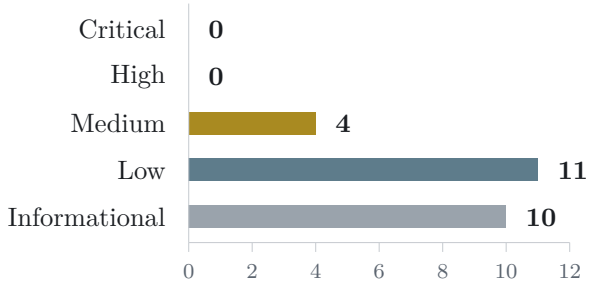
Did accounting and solvency invariants hold? Yes, for every invariant this review was able to make executable. Price-per-share is preserved exactly across mint, redeem, flash mint, flash redeem and fee settlement; bond collateral reconciles to the sum of tranche claims; rollover is value-neutral to within documented ceiling-rounding dust. See §5.

Do unresolved centralized or economic risks remain? Yes, and they are material. The owner can upgrade every proxy, repoint each note's controller, install an arbitrary fee policy and balancer, and set an AUM fee that dilutes holders essentially without limit. The keeper can pause user operations indefinitely and set the value cap. None of these is timelocked in code. See §10.

Readiness The contract logic is in good shape for a mainnet deployment *once* MED-01 through MED-04 are resolved or explicitly accepted in writing, the fee-curve parameters are chosen with the freeze behaviour in mind, and the privileged roles sit behind a timelock. This report makes no claim that the system is secure; it reports what was and was not found.

2.5 Findings by severity

Severity	Count	Confirmed by proof-of-concept
Critical	0	—
High	0	—
Medium	4	4
Low	11	7
Informational	10	1
Total	**25**	**12**



Findings by severity. Bar shades are ordered by luminance so the chart reads correctly in greyscale.

3 Scope and methodology

3.1 In-scope files

58 Solidity files, 11,001 physical lines, in two packages:

Package	Files	Lines	Contents
`core-contracts/`	35	6,411	bonds, tranches, perp notes, controller, fee policy, balancer, oracles, router
`vaults/`	23	4,590	depositor vaults, single-asset routers, DEX strategies, CoW broker

Full inventory with per-file review status: Appendix A.

3.2 Out of scope

`ops/` (Go keeper), `subgraph/`, `research/`, Ignition deployment modules, Hardhat tasks, and every external dependency. Three of these were read for context and are referenced where a finding depends on them — the subgraph's event handlers (INFO-01), the keeper's liveness role (MED-03), and the deployment modules' parameter choices (LOW-07) — but none was assessed.

3.3 Environment and reproduction

Node.js	v22.15.0 (project requires ≥22; `.nvmrc` pins 22)
Package manager	yarn 4.9.1, `nodeLinker: node-modules`
Hardhat	2.29.0, in-process EDR network, hardfork `cancun`
solc	0.8.35+commit.47b9dedd, optimizer on (800 runs), `viaIR`
ethers / mocha / chai	6.16.0 / 11.7.6 / 4.5.0
solidity-coverage	0.8.17
OpenZeppelin	`contracts` 5.6.1, `contracts-upgradeable` 5.6.1, `hardhat-upgrades` 3.9.1
Slither	0.11.6
Host	macOS 15.7.7, x86_64

All work was performed in an isolated copy of the repository so the reviewed working tree was never modified. The copy shares `node_modules` by symlink; every added file lives under `test/audit/` in the copy and is delivered separately. Exact commands are in Appendix C.

3.4 Method

The review ran in phases, each feeding the next.

1. Build and baseline. Reproduce the build, run both existing suites, record warnings, failures and skips. Result: 1,638 core tests and 838 vault tests, all passing, no skips. Compiler warnings are confined to OpenZeppelin dependencies (an `at` identifier deprecation and the standard EIP-1153 transient-storage composability notice); no in-scope file produced a warning.

2. System reconstruction. Rebuild the protocol's structure from source rather than from documentation: contract topology, storage layout, initialization, roles, custody points, oracle inputs, cross-contract call paths, callback surfaces, and the state machines for bonds, rollover, rebalance and vault sessions. The asset-flow map in §4.2 is the output.

3. Threat model. Enumerate actors and capabilities, and for each trusted party separate what the code prevents from what it merely assumes (§4.4).

4. Invariant catalogue. Write down the properties the system claims, before testing, then make each one executable where practical (§5). Twenty-two invariants were catalogued; sixteen were exercised by an executable test, four were verified by symbolic derivation over the source, and two could not be reduced to a test and are reported as such.

5. Line-by-line manual review. Every in-scope line, plus the inherited OpenZeppelin behaviour that matters (`ERC20Upgradeable`, `Ownable2StepUpgradeable`, `ReentrancyGuardTransient`, `EnumerableSet`, `Clones`, `TransientSlot`, `SafeERC20`, `Math.mulDiv`). Call paths were traced whole rather than function by function.

6. Automated analysis. solhint with the project's own configuration (clean on both packages), solidity-coverage re-measured from scratch, Slither 0.11.6, and compiler warning capture. Every automated result was manually triaged; nothing from a tool appears in this report as a finding without independent confirmation.

7. Adversarial testing. Twenty-three proof-of-concept tests written against the repository's own Hardhat harness, covering share-price manipulation by donation, fee-curve denial of service, oracle unavailability, privileged-role dilution, rounding extraction, and accounting drift under a non-canonical fee policy. All are delivered and all pass.

8. Candidate discipline. Every hypothesis was recorded and then attacked. Thirty-one candidates were opened: 16 became findings — 12 of those confirmed with a proof-of-concept and 4 by source reading — and 15 were invalidated. The remaining nine findings (INFO-02 through INFO-10) came out of the line-by-line pass rather than from a hypothesis under test. The invalidated set is published in Appendix D so the same ground does not have to be re-walked.

3.5 Adversarial strategies applied

Reentrancy (same-function, cross-function, cross-contract, read-only, and via token callbacks on transfer, mint, burn and fee payout); flash-funded mint / redeem / rebalance sequencing, including whether the reentrancy guard is per-call rather than per-transaction; first-depositor and donation-based share inflation on both the notes and the vaults; repeated-rounding extraction on tranche redemption; deposit/withdraw and mint/redeem round trips; forced value transfer between the two notes by timing a permissionless rebalance; settlement-price selection by timing a permissionless `mature()`; oracle staleness, zero and discontinuity; L2 sequencer downtime; front-running of `upkeep`-gated router paths; gas growth in the reserve loops; storage collision across the ERC-7201 namespace; implementation-contract initialization; arbitrary-approval abuse in the stateless routers; and solver-controlled inputs into the CoW broker's pre-hook.

3.6 Material limitations

These are the parts of the assessment a reader should discount.

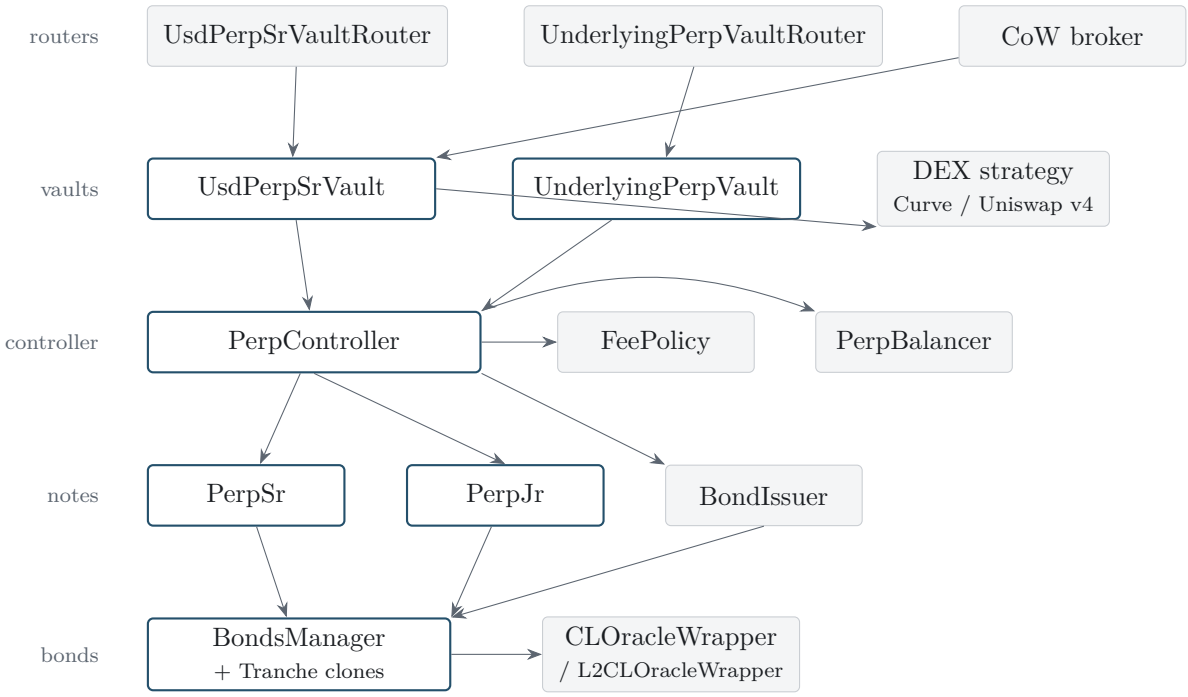
- **No new fuzz campaign was run.** The repository ships property and path fuzz suites under `core-contracts/test/fuzz/` which are disabled by default (`FUZZ_RUNS=0`); they were not executed at scale here, and no new stateful multi-actor invariant campaign was authored. Section 5 therefore reports directed tests, not fuzzed ones. This is the single largest gap in this engagement and is the first thing a follow-up should close.

- **No fork testing.** The vault fork suites under `vaults/test/fork/` require an `ALCHEMY_SECRET` that was not available. Curve pool behaviour, Uniswap v4 pool behaviour, Morpho Blue flash-loan mechanics and CoW settlement were therefore reviewed against their interfaces and the repository's mocks, never against live deployments. Any conclusion about those integrations inherits the mocks' fidelity.
- **No testnet interaction.** No transactions were submitted to any network. No testnet deployment, RPC endpoint, sender account or budget was authorised for this engagement, so the authorisation and preflight procedures for live interaction were not exercised. Everything reported here ran on a local in-process EVM.
- **Economic conclusions are unpriced.** Statements about whether an attack is profitable use the fee parameters in `FeePolicy.init()` and the vault defaults. No liquidity depth, borrow cost or gas price data was available, so extraction estimates are given as ratios of notional rather than dollars.
- **Deployment configuration was not reviewed.** Ignition modules, constructor arguments, initial owner and keeper addresses, oracle addresses, `STALE_AFTER` values and pool choices are out of scope, and several findings' severity depends on them.
- **Slither results are reported as tool output.** Where Slither's detectors and this review disagree, the manual reading is what is reported; see §11.

4 Architecture and trust model

4.1 System overview

Four layers, each trusting only the one below it.



Contracts drawn with a heavy border custody user value. Everything else is stateless policy, plumbing, or a price source.

4.2 Asset flow and custody

Value enters at four doors and lives in exactly four places.

Custody point	Holds
`BondsManager`	All underlying collateral, pooled across every bond it has ever issued. Per-bond accounting is `underlyingBal` pre-maturity and `srClaim + jrClaim` post-maturity.
`PerpSr`	Senior tranche tokens of several maturities; transiently underlying between rollovers; junior tranches only after a Jr→Sr rebalance.
`PerpJr`	Junior tranche tokens plus the idle underlying that funds rollovers and flash redemptions.
`PerpController`	Withheld operation fees only, split by the `protocolFeeBals` mapping into a protocol share (withdrawable by the fee collector) and a holder share (settled into PerpJr).
`UsdPerpSrVault`	USD + PerpSr, plus a transient "resting" underlying balance between trades.
`UnderlyingPerpVault`	Underlying + one perp note.
Routers, broker, strategies	By design, nothing between transactions.

The paths, in the direction value moves:

1. **In, tranched:** user → `BondsManager.deposit` → senior + junior tranches → `PerpController.mint2` → tranches into a note, notes to the user.
2. **In, flash:** user → `flashMint2` with underlying → underlying lands directly in a note, notes minted against the note's live price-per-share, and a "lazy rollover" may immediately convert the PerpSr side into fresh tranches using PerpJr liquidity.
3. **Out, in kind:** `redeem2` burns notes and hands back a pro-rata slice of the note's whole reserve: underlying and every tranche.
4. **Out, in underlying:** `flashRedeem2` burns notes, sweeps PerpSr's reserves into PerpJr to cover the senior leg, and pays out of PerpJr's underlying. This is the only exit that is liquidity-constrained, and the only one the vaults use.
5. **Between the notes:** `rebalance()` moves reserve value from the overweight note to the underweight one, sized by `FeePolicy.computeRebalanceVal` and drawn in maturity order. `rollover()` cycles stale senior tranches through PerpJr into fresh ones and is designed to be value-neutral to both notes.
6. **Fees:** withheld in the operated asset, held by the controller, then either pulled by the fee collector (protocol share) or burned into PerpJr (holder share). AUM fees are a dilution mint at rebalance, not a transfer.

Where recorded accounting can diverge from balances

Four places, all identified and none found to be exploitable in a seeded system:

- `PerpTranche.reserveData()` reads live `balanceOf` for every *registered* tranche, so a direct ERC-20 transfer to a note counts as reserve without minting supply. This is the mechanism behind MED-02.
- A tranche that is *not* registered and is transferred to a note is invisible to `reserveData()` and is never swept (INFO-02).
- `collectedFeeBal(token)` is `balanceOf(this) - protocolFeeBals[token]`. It is a subtraction on an unvalidated pair, and a fee policy that over-reports the protocol share makes it underflow permanently (LOW-04).
- `UsdPerpSrVault.getTVL()` deliberately excludes the resting underlying balance, while `computeMintAmt` prices it in. Deposits pay for it; redemptions do not receive it (LOW-06).

4.3 Roles and privileges

Role	Where	Can do
Owner	every upgradeable contract, via `Ownable2StepUpgradeable`	Upgrade the proxy; repoint each note's controller; install an arbitrary fee policy, balancer and issuer; set DR targets, rebalance cadence, liquidity floors, oracle reference; set AUM and protocol fee percentages and the fee collector; whitelist bond assets, issuers and oracles; whitelist vault swappers; replace the vault DEX strategy; seed the vaults
Keeper	`PerpController`, both vaults	Pause and unpause system ops, user ops, and each vault; set `valueCap`
Fee collector	`PerpController`	Withdraw the accrued protocol fee share
Issuer	`BondsManager`	Create bonds
Controller	`PerpSr`, `PerpJr`	Mint, burn, move reserves, register tranches, recover
Swapper	`UsdPerpSrVault`	Call the passive `swap` leg
CoW solver	via the broker	Drive a settlement that calls `provideInventory` inside a flash loan
Anyone	`PerpController`, vaults, issuer	`rebalance`, `rollover`, `upkeep`, `issue`, `cleanupMature`, `mature`, vault `rebalance` and `upkeep`

4.4 Trust boundaries

For each trusted party, what the code prevents versus what it merely assumes:

Party	Prevented by code	Only assumed	Worst case if compromised
Owner	Nothing meaningful. Two-step transfer prevents an accidental handover, not a deliberate one.	Honest parameter choice; that a timelock or multisig sits in front (the prior review recorded a documented multisig, which this review could not verify on-chain).	Total. Upgrade to arbitrary code, or without upgrading: set the AUM fee to 99% and take 99% of both notes (MED-04), or repoint `PerpTranche.updateController` to an attacker contract and drain both reserves.
Keeper	Cannot move funds.	That pauses are used for incidents, not censorship.	Indefinite freeze of all user operations; `valueCap` set to 0 to block minting.
Fee collector	Cannot withdraw more than `protocolFeeBals`.	—	Bounded to accrued fees.
Oracle operator	Zero and stale answers are rejected; the L2 wrapper adds a sequencer check and grace period.	Feed correctness; that `STALE_AFTER` matches the feed's real heartbeat; that a single feed is enough.	Wrong price → wrong tranche split at settlement and wrong note valuation. No price is a full protocol freeze (MED-03).
Fee policy	`totalFeePerc >= ONE` reverts.	That `protocolFeePerc <= totalFeePerc`. The explicit guard for this was added and then deliberately removed in the reviewed commit.	Permanent brick of fee settlement and `rebalance()` (LOW-04).
Balancer	Controller re-reads storage after a rollover.	That the returned plan conserves value in each note.	Value transfer between the notes.
Vault swapper	Price floor, direction, DR gap, session budget and minimum size are all enforced on-chain against the oracle.	Nothing material.	Bounded to `oracleDeviationPerc` of the session budget.
CoW solver	Broker holds no capital; every leg is price-checked by the vault; the loan must be repaid or the transaction reverts.	Morpho, CoW settlement and the flash-loan router behave as documented.	Bounded to `oracleDeviationPerc` of the session budget.
Underlying token	—	Non-rebasing, non-fee-on-transfer, no transfer hooks. Stated in NatSpec; not enforced anywhere.	Fee-on-transfer or rebasing collateral breaks bond accounting outright.

4.5 Upgrade model

Every stateful contract except `Router`, the DEX strategies, the CoW broker and the oracle wrappers is deployed behind an OpenZeppelin transparent proxy and initialised through an `init` function; implementations call `_disableInitializers()` in their constructors, so implementation-contract takeover is not available. `PerpTranche` uses ERC-7201 namespaced storage at a fixed slot, which keeps its layout independent of the inherited OpenZeppelin storage, a good choice, and it means the notes can be upgraded without layout risk. The other upgradeable contracts use ordinary sequential storage; `UnderlyingPerpVault`'s layout was deliberately repacked in the reviewed commit, which is safe only because nothing is deployed behind a proxy yet. That window closes at first deployment.

4.6 Oracle model

One `IPriceOracle` per whitelisted asset, registered on `BondsManager`. `CLOracleWrapper` wraps a Chainlink aggregator, rejects non-positive answers, rejects `updatedAt == 0`, rejects answers older than an immutable `STALE_AFTER`, and scales to 18 decimals. `L2CLOracleWrapper` adds a sequencer uptime feed and a grace period. Both inherit `CachedOracle`, which memoises the price in transient storage for the duration of a transaction so every read inside one operation sees the same number, a genuinely good design choice that removes a whole class of intra-transaction inconsistency.

The model's weakness is that there is exactly one feed per asset, no fallback, and no notion of a degraded mode. See MED-03.

4.7 State machines

Bond *created* → *live* (accepts deposits and proportional redemptions, valued continuously against the current oracle price) → *settled* (claims frozen at the price observed when `mature()` was first called; each tranche redeems independently). Transitions are one-way. `mature()` is permissionless and has no deadline (LOW-09).

Rebalance Gated by `onlyRebalanceTime`: at most once per `rebalanceFreqSec`, and only inside a window of `rebalanceWindowLengthSec` opening `rebalanceWindowOffsetSec` into each period. Permissionless within that window.

Rollover Two policies. The primary distributes value evenly across all fresh senior tranches and requires enough PerpJr liquidity to pair each stale senior with its junior counterpart; if that is short it scales down to the paired subset. The backup deploys whatever free liquidity exists into the single fresh bond whose CDR is closest to 1.0.

Vault session Each `rebalanceFreqSec` window opens a session that locks a direction and a budget of `maxDeploymentVelocityPerc` of TVL. The active (DEX) leg can be delayed behind the passive (swapper) leg by `activeRebalanceDelaySec`.

4.8 Off-chain assumptions and liveness dependencies

The protocol needs somebody to call `upkeep()` so bonds settle and fresh ones are issued, `rollover()` so PerpSr's exposure does not decay into cash, and `rebalance()` so the two notes track their target ratio. All three are permissionless, which is the right design, but nothing pays for them, so in practice liveness rests on the Go keeper under `ops/`. Two consequences worth stating: a keeper outage lets bonds sit unsettled and re-price against a moving market (LOW-09), and it lets stale tranches accumulate in PerpSr, which degrades the senior leg's exposure rather than breaking it.

5 Security properties tested

Each invariant was written down before it was tested. *Method* distinguishes the repository's own suite from tests added for this review, and from properties established by derivation over the source. *Result* is what was actually observed.

ID	Property	Contracts	Method	Result
INV-01	`redeem2` leaves reserve-value-per-note exactly unchanged: burning n notes removes $(n - \text{fee})/S$ of reserves and re-mints the fee, so R/S is invariant.	PerpController, PerpHelpers	Derivation + existing suite	Held
INV-02	`mint2` leaves reserve-value-per-note exactly unchanged, fee included.	PerpController, PerpHelpers	Derivation + existing suite	Held
INV-03	`flashMint2` / `flashRedeem2` price against the same pre-operation snapshot and preserve R/S.	PerpController	Existing suite	Held
INV-04	Fee settlement is neutral to PerpSr holders and accretive only to PerpJr holders; it never mints notes to anyone.	PerpController	Existing suite	Held
INV-05	At settlement, `srClaim + jrClaim` equals the bond's whole collateral; neither can be negative.	BondsManager	Existing suite	Held
INV-06	The pooled collateral in `BondsManager` covers every bond's outstanding claims simultaneously, including bonds settling in opposite directions.	BondsManager	Existing suite	Held
INV-07	Rollover is value-neutral to both notes, up to $O(1)$ wei per fresh tranche from deliberate ceiling rounding.	PerpBalancer, PerpController	Derivation + existing suite	Held (not fuzzed)
INV-08	Redemption returns a strictly pro-rata slice of every reserve asset; no cherry-picking.	PerpHelpers	Existing suite	Held
INV-09	The protocol fee share never exceeds the total fee withheld.	PerpController, FeePolicy	**Added PoC**	**Not enforced** — LOW-04
INV-10	Only the controller can mint, burn, move reserves or register tranches on a note.	PerpTranche	Existing suite	Held
INV-11	Initialization runs once, on the proxy; implementations are locked.	all upgradeable	Existing suite	Held
INV-12	Only the BondsManager can mint or burn a tranche token.	Tranche	Existing suite	Held
INV-13	A note's tracked tranche set never exceeds `maxTrancheCount()`; zero-balance entries are pruned.	PerpTranche	Existing suite	Held
INV-14	Deviation ratios fed to the fee curve stay inside the protocol's own saturation ceiling.	PerpController, FeePolicy	**Added PoC**	**Fails for vmDR** — LOW-01
INV-15	A holder can never redeem more than their pro-rata claim, at any size including one wei.	PerpController	Existing suite	Held
INV-16	After seeding, a vault's note supply is permanently non-zero (`MINIMUM_LIQUIDITY` locked in the vault).	both vaults	Existing suite	Held
INV-17	A vault deposit cannot move the vault's asset mix; it is ratio-bound and reads no oracle for pricing the pair.	both vaults	Existing suite	Held
INV-18	Neither vault trades at worse than the oracle price adjusted by `oracleDeviationPerc`, in either direction and on both the active and passive legs.	UsdPerpSrVault	Existing suite	Held

continued overleaf

Security properties tested — continued

ID	Property	Contracts	Method	Result
INV-19	The CoW broker holds no capital across settlements; any surplus is swept to the vault and any shortfall reverts the settlement.	UsdPerpSrVaultCowBroker	Existing suite	Held
INV-20	A vault deposit never lowers the value behind an existing note, measured on the whole balance sheet including resting underlying.	UsdPerpSrVault	Existing suite	Held
INV-21	A vault redemption returns the redeemer's full pro-rata share of vault value.	UsdPerpSrVault	**Added PoC**	**Fails** — LOW-06
INV-22	Every value-moving entry point stays callable while the system is solvent — in particular, a holder can always exit.	PerpController	**Added PoC**	**Fails** — MED-01, MED-03

Notes and limits. INV-07 is the one invariant this review would most like to see fuzzed: the rollover plan has two policies, a scale-down branch, and ceiling rounding in three places, and directed tests cover each branch but not their interaction across long sequences. INV-01 through INV-04 were additionally derived symbolically, which is stronger than the tests for those particular properties: burning n notes while returning $(n - f)/S$ of reserves and re-minting f leaves $R(1 - (n - f)/S)/(S - n + f) = R/S$ identically, for all n and f, with no rounding term.

6 Findings summary

ID	Title	Severity	Conf.	Status	Component
MED-01	Fee curve doubles as a hard gate; one-sided redemption is frozen at ordinary deviation levels	Medium	High	Open	PerpController, FeePolicy
MED-02	Perpetual notes have no first-depositor protection; a tranche donation zeroes the next mint	Medium	High	Open	PerpTranche, PerpHelpers
MED-03	One price feed gates every entry point, exits included	Medium	High	Open	BondsManager, oracles
MED-04	AUM fee is unbounded and charged per call; one rebalance can dilute holders to 1%	Medium	High	Open	FeePolicy, PerpController
LOW-01	`vmDR` is not clipped; the fee curve overflows `int256` and panics	Low	High	Open	PerpController, LineHelpers
LOW-02	`deposit` takes collateral and mints zero tranches below one unit of value	Low	High	Open	BondsManager
LOW-03	`redeemImmature` burns senior without the paired junior; dust moves Jr→Sr	Low	High	Open	BondsManager
LOW-04	Controller trusts the fee policy's protocol-share ordering; a violation bricks settlement	Low	High	Open	PerpController
LOW-05	`upkeep()` reverts `NoOp`, making the routers' upkeep flags unusable	Low	High	Open	PerpController, routers
LOW-06	Vault charges depositors for resting underlying but never pays redeemers for it	Low	High	Open	UsdPerpSrVault
LOW-07	`updateTargetDR` accepts a zero lower bound, disabling de-risking at DR = 0	Low	High	Open	both vaults
LOW-08	Stateless routers grant unbounded approvals to caller-supplied addresses	Low	Medium	Acknowledged	Router, vault routers
LOW-09	`mature()` is permissionless with no deadline; the caller picks the settlement price	Low	Medium	Open	BondsManager
LOW-10	`updateBondConfig` under-validates; a bad value reverts issuance and everything downstream	Low	High	Open	BondIssuer
LOW-11	Rollover divides by `valueRatio` with no zero guard	Low	Medium	Open	PerpBalancer
INFO-01	Fee settlement moves underlying with no `ReserveSynced`; the indexer misses it	Informational	High	Open	PerpController
INFO-02	Unregistered tranches sent to a note are invisible and permanently stranded	Informational	High	Open	PerpTranche
INFO-03	`computeMintAmt` divides by zero when both vault legs are empty	Informational	Medium	Open	UnderlyingPerpVault

continued overleaf

Findings summary — continued

ID	Title	Severity	Conf.	Status	Component
INFO-04	`cachePrice()` and `resetCache()` are permissionless	Informational	High	Open	CachedOracle
INFO-05	`answeredInRound < roundId` is a deprecated staleness check	Informational	High	Open	CLOracleWrapper
INFO-06	Reserve reads scale with tranche count and are repeated 2–3× per operation	Informational	Medium	Open	PerpTranche, PerpController
INFO-07	`Router.redeem3` sweeps the router's whole balance to the caller	Informational	High	Open	Router
INFO-08	`_enforceMintCap` ignores `vmDR` and rejects every mint at `subDR == 1`	Informational	High	Open	PerpController
INFO-09	No privileged setter emits an event; two contracts emit none at all	Informational	High	Open	all packages
INFO-10	Address setters accept the zero address	Informational	High	Open	PerpController, vaults

Status is this review's own assessment at the reviewed commit. *Acknowledged* marks a finding the source already documents as a known property. No maintainer responses or fixes were supplied during this engagement, so no retest column is populated.

7 Detailed findings

7.1 MED-01 — The fee curve is a hard gate, and it closes on redemptions

Severity	Medium
Confidence	High — confirmed by proof-of-concept
Status	Open

Affected `core-contracts/contracts/PerpController.sol` `_computeOpFees` (L1288–L1304), `redeem2` (L574), `flashRedeem2` (L795); `core-contracts/contracts/strategies/FeePolicy.sol` `init` (L124–L147), `computeOpFees` (L250–L277).

Category Denial of service / frozen funds; broken availability invariant INV-22.

Broken property A solvent holder can always exit their position.

Description. `FeePolicy.computeOpFees` sums a base fee and two impact fees, each of which is the *average* of a three-piece linear curve over the interval the operation moves a deviation ratio across. The controller then treats the result not only as a price but as a gate:

```
1301 if (fees.totalFeePerc >= ONE) {
1302   revert OpDisabled();
1303 }
```

The curves installed by `FeePolicy.init()` reach exactly 100% inside the ordinary operating range. `subDRUp` is $[(0,0),(0.95,0),(1.2,0.05),(2.0,1.0)]$. On its third segment the slope is $0.95/0.8 = 1.1875$ per unit of ratio, so $f(x) = 0.05 + 1.1875\,(x - 1.2)$, which reaches exactly 1.0 at $x = 2.0$ and is extrapolated linearly above it. Redeeming PerpSr lowers the senior leg's TVL, which *raises* `subDR`, so the redemption selects the up curve, and the average it pays over even an infinitesimal interval is $f(\text{subDR})$. With the default base fee of zero, that reaches 100% at exactly subDR $= 2.0$.

The result is that at a subscription deviation ratio of 2.0 or above, PerpSr cannot be redeemed at all, in any size. The mirror case holds on the other side: `subDRDown` is $[(0.66,1.0),(0.9,0.05),(1.2,0),(2,0)]$, whose first segment extrapolated below 0.66 exceeds 1.0, so at subDR ≤ 0.66 PerpJr cannot be redeemed — and a larger junior redemption, whose interval straddles that point, is refused from above it. Junior deviation ratios fall precisely when the junior buffer is being consumed — a market crash — so the junior freeze arrives at the worst moment.

The freeze is self-reinforcing. The only operations that reduce a high `subDR` are PerpSr mints and PerpJr redemptions; a rational actor will not deposit into a leg they have watched become unredeemable, so the state can persist until the owner replaces the fee policy.

Preconditions. `subDR` at or above 2.0 for the senior freeze, at or below 0.66 for the junior freeze, with the shipped default curves. `subDR` is $\frac{\text{PerpJr TVL}}{\text{PerpSr TVL}}$/targetSystemRatio, so with a target of 2.0 the senior freeze begins once the junior leg is four times the senior leg by value, a level a strong rally in a leveraged junior tranche reaches without help.

Attack sequence. No attacker is required; this is reached by market movement. An attacker who wants to *induce* it must push `subDR` across the threshold themselves, paying the same impact fees, so the finding is reported as a design and availability defect rather than an exploit.

Impact. Frozen redemptions on one leg of the protocol, for an unbounded duration, with no in-contract override. The blast radius extends into the vault layer: `UsdPerpSrVault` exits its PerpSr position exclusively through `flashRedeem2`, which computes its fee from the same curves, so `UsdPerpSrVaultRouter.redeemUsd` reverts as well. Vault LPs can still burn notes for raw PerpSr, but cannot convert it.

Proof of concept. `proofs-of-concept/core/MED-03-op-disabled-freeze.ts`. Builds a book with 100,000 senior and 800,000 junior tranche units through a zero-fee policy, switches on the shipped curves, and observes:

```
subDR = 4.0  vmDR = 1.333333333333333333
PerpSr redemption: reverted OpDisabled. PerpJr redemption: succeeded.
every Sr exit size tried (100%, 1%, 0.01%) reverted OpDisabled
subDR = 0.3
PerpJr redeem2 reverted OpDisabled at subDR = 0.3
```

Reproduction.

```
cd core-contracts
FUZZ_RUNS=0 PATH_FUZZ_RUNS=0 npx hardhat test test/audit/MED-03-op-disabled-freeze.ts
```

Root cause. Two decisions compose badly. The fee function is allowed to exceed 100% because each impact term is independently clipped to `[0, ONE]` and then summed with the base fee; and the controller converts "fee $\geq$ 100%" into a revert rather than into a clamp. Neither decision distinguishes an operation that worsens the system (which arguably should be discouraged) from an operation that lets a holder leave (which arguably should never be blocked).

Recommendation. Decouple pricing from availability, in this order of preference:

1. Clamp rather than revert. Cap `totalFeePerc` at some `maxOpFeePerc` strictly below `ONE` inside `computeOpFees`, so an extreme deviation makes exiting expensive rather than impossible.
2. If a gate is genuinely wanted, gate only the operations that grow the imbalance — mints — and never redemptions. `OpFeeInput` already carries enough context (`isPerpSrOp`, and pre/post ratios) to distinguish them.
3. Failing both, bound the curve shape at configuration time: reject a `Linear3PFn` whose extrapolation can exceed `ONE` within the representable ratio range, so an operator cannot install a freezing curve by accident.

   Sketch of option 1:

```
// FeePolicy
uint64 public maxOpFeePerc; // e.g. 0.20e18, set by updateProtocolFeeConfig

f.totalFeePerc = Math.min(rawTotal, maxOpFeePerc);
f.protocolFeePerc = Math.min(f.totalFeePerc, protocolMaxOpFeePerc);
```

Regression test. Assert that `redeem2` and `flashRedeem2` succeed for both legs across a sweep of `subDR` from 0.1 to 10 and `vmDR` from 0.5 to 5, and that the realised fee is monotone in deviation but never reaches `ONE`.

If accepted as designed. Document the freeze thresholds in user-facing material, publish the live `subDR` and the distance to each threshold, and treat the fee-curve parameters as a safety parameter rather than a revenue one.

7.2 MED-02 — Perpetual notes have no first-depositor protection

Severity	**Medium**
Confidence	High — confirmed by proof-of-concept
Status	Open

Affected `core-contracts/contracts/_utils/PerpHelpers.sol` `computeMintAmt` (L25–L43), `computeFlashMintAmt` (L81–L87); `core-contracts/contracts/perps/PerpTranche.sol` `reserveData` (L241–L262).

Category Theft / loss of user funds; share-price manipulation.

Broken property Depositing value into a note always mints a claim on that value.

Description. `computeMintAmt` bootstraps at 1:1 when supply is zero and prices against live reserves afterwards:

```
return
  perpSupply > 0
    ? rtd.data.value(trAmtIn.amount).mulDiv(perpSupply, reserveValue_)
    : trAmtIn.amount;
```

`reserveValue_` is derived from `PerpTranche.reserveData()`, which reads the note's live `balanceOf` for every *registered* reserve tranche. The tranche tokens are plain ERC-20s, and the current bond's tranches are registered on every `mint2`, so an ordinary `transfer` into the note raises `reserveValue` without minting any supply. That is the classic ERC-4626 inflation primitive, and there is nothing here to blunt it: no virtual shares, no decimal offset, no locked minimum, and no floor on the first mint.

The vault layer immediately above defends exactly this case: both `UnderlyingPerpVault` and `UsdPerpSrVault` mint and permanently lock `MINIMUM_LIQUIDITY` notes on their seed deposit, with a comment saying so. The asymmetry is what makes this look like an oversight rather than a decision.

Preconditions. Note supply at or near one wei. That is the state immediately after deployment and before the protocol seeds the notes, and it is reachable again if a note is ever redeemed down to dust.

Attack sequence.

1. Attacker deposits one wei of the current bond's senior tranche through `mint2`; supply becomes 1.
2. Attacker transfers their remaining senior tranche balance directly to the `PerpSr` contract. Reserve value jumps; supply stays at 1.
3. Victim calls `mint2` with a real deposit. `value * 1 / reserveValue` floors to zero: the tranches are pulled in and zero notes are minted.
4. Attacker redeems their single wei and receives the entire reserve.

Impact. Total loss of the victim's deposit. The same rounding also zeroes `flashMint2`, which matters because the vault layer calls it with `minPerpSrAmtOut = 0` and checks only the returned fee percentage; see `PerpVaultHelpers.flashMintPerp` and `UnderlyingPerpVault.rebalance`. A vault rebalancing into a manipulated note would surrender its underlying for nothing.

Attacker cost and benefit. Cost: gas, plus the tranche balance donated in step 2, which is recovered in full in step 4. Benefit: the victim's entire deposit. Repeatable while supply stays at one wei. The attack is atomic if steps 1–2 and step 4 are placed either side of the victim's transaction in the same block.

Proof of concept. `proofs-of-concept/core/MED-02-perp-donation-inflation.ts`:

```
perpSr supply after attacker's 1-wei seed = 1
after donation: supply=1  reserveValue=500000000000000000000
victim deposits 12500000000000000000000 sr-tranche -> mints 0 perpSr
attacker redeems 1 wei perpSr -> receives 125125000000000000000000000 sr-tranche
flashMint2 -> 0 perpSr
```

Existing partial mitigations. `mint2` and `flashMint2` both accept a `minPerpSrAmtOut` / `minPerpJrAmtOut`, and a caller who sets them protects themselves. The core `Router` passes them through. The vault layer does not use them.

Root cause. Supply can legitimately be arbitrarily small, and reserve value is read from live balances rather than from an internal accounting variable.

Recommendation. Give the notes the same defence the vaults already have. Either:

```
// PerpTranche
uint256 public constant MINIMUM_LIQUIDITY = 1e12;

// in the controller's first-mint path, when perp.totalSupply() == 0:
perp.mintTo(address(perp), MINIMUM_LIQUIDITY);   // permanently locked
perp.mintTo(msg.sender, perpAmtMint - MINIMUM_LIQUIDITY);
```

or apply a virtual-shares offset inside `computeMintAmt` and `computeRedemptionAmts` as OpenZeppelin's ERC-4626 does. Independently, have the vault layer pass a non-zero `minPerpAmtOut` derived from its own pre-trade snapshot rather than 0, so it can never be the victim of step 3.

Regression test. Assert that after the seed, no sequence of donation plus mint can leave a depositor with zero notes for a non-zero deposit, and that `perp.totalSupply()` never returns below `MINIMUM_LIQUIDITY`.

7.3 MED-03 — One price feed gates every entry point, exits included

Severity	Medium
Confidence	High — confirmed by proof-of-concept
Status	Open

Affected `core-contracts/contracts/bonds/BondsManager.sol` `assetPrice` (L540–L546), `trancheData` (L629–L689); `core-contracts/contracts/perps/PerpTranche.sol` `reserveData` (L241–L262); `core-contracts/contracts/oracles/CLOracleWrapper.sol` (L40–L63); `core-contracts/contracts/oracles/L2CLOracleWrapper.sol` (L50–L62).

Category Protocol-wide denial of service; liveness dependency on a single external system.

Broken property A holder can exit while the system is solvent (INV-22).

Description. `trancheData` prices any unmatured bond against the live oracle. It has no fallback:

```
540 function assetPrice(IERC20 asset) public view override returns (uint256) {
541   IPriceOracle oracle = oracles[asset];
542   if (address(oracle) == address(0)) {
543     revert UndefinedOracle();
544   }
545   return oracle.getValidPrice();
546 }
```

`PerpTranche.reserveData()` calls `tranche.data()` for every registered tranche, and every `PerpController` entry point begins with `systemReserveState()`. A single revert inside `_fetchPrice` therefore propagates to `mint2`, `redeem2`, `flashMint2`, `flashRedeem2`, `rebalance`, `rollover`, `upkeep`, and even the view `systemReserveState()` that off-chain systems read.

`_fetchPrice` reverts on four conditions: a non-positive answer, an `updatedAt` of zero, an answer older than the immutable `STALE_AFTER`, and `answeredInRound < roundId`. On an L2 the wrapper adds two more: the sequencer feed reporting down, and the whole of the configured grace period after it comes back up.

Preconditions. Any of: a Chainlink feed exceeding its heartbeat, a feed reporting zero or a negative answer, an aggregator migration, or — on an L2 — a sequencer restart. None requires an attacker.

Impact. The entire protocol halts, including withdrawals, for as long as the feed is unavailable plus (on an L2) the grace period. Holders cannot exit into a falling market. The vaults inherit the halt because every vault path reaches the controller. Note that the freeze is symmetric and does not itself lose value; it is an availability failure, not a theft. But for a leveraged product, an enforced inability to exit during a price move is a material loss of optionality.

Proof of concept. `proofs-of-concept/core/MED-04-oracle-dos.ts` removes the feed's answer after establishing a live position:

```
redeem2                  reverted: true
flashRedeem2             reverted: true
mint2                    reverted: true
rollover                 reverted: true
upkeep                   reverted: true
```

```
systemReserveState (view)  reverted: true
PerpSr.reserveData() and PerpJr.reserveData() both revert
```

Root cause. Valuation and availability are coupled. Every operation, including ones that need no fresh price to be safe, is priced from the same live read.

Recommendation.

1. **Add a fallback source.** A second aggregator, or a last-good-price cache with a bounded age and a deviation limit, so a single feed outage degrades rather than halts.
2. **Add an emergency in-kind exit.** `redeem2` returns a pro-rata slice of reserves; that operation does not actually need a price to be safe: it needs one only to compute a fee and a slippage figure. A guardian-gated or automatically-enabled "zero-fee proportional redemption" path that skips `systemReserveState()` entirely would let holders leave during an outage without any pricing at all.
3. **Make `STALE_AFTER` governable** rather than immutable, so a heartbeat change on the feed does not require redeploying the wrapper and re-registering it on every bond.
4. **Publish the assumption.** `STALE_AFTER` and the grace period should be documented per deployment alongside the feed's actual heartbeat.

Regression test. With the feed reverting, assert that the emergency exit path still returns each holder's pro-rata reserve slice and that no other state-mutating path is callable.

7.4 MED-04 — The AUM fee is unbounded and charged per call, not per unit time

Severity	Medium
Confidence	High — confirmed by proof-of-concept
Status	Open

Affected `core-contracts/contracts/strategies/FeePolicy.sol` `updateAumFeePerc` (L229–L238); `core-contracts/contracts/PerpController.sol` `_chargeAUMFees` (L1135–L1152), `rebalance` (L423–L444).

Category Privilege escalation / unbounded governance capability.

Broken property Privileged parameters have bounds proportionate to their effect.

Description. The AUM fee setter's only constraint is that each percentage is strictly below 100%:

```
229 function updateAumFeePerc(
230   uint64 perpSrAUMFeePerc_,
231   uint64 perpJrAUMFeePerc_
232 ) external onlyOwner {
233   if (perpSrAUMFeePerc_ >= ONE || perpJrAUMFeePerc_ >= ONE) {
234     revert InvalidPerc();
235   }
236   ...
237 }
```

The controller then charges it as a dilution mint whose multiplier diverges as the percentage approaches one:

```
1143 uint256 srAumFeeAmt = perpSrSupply.mulDiv(srFeePerc, ONE - srFeePerc);
1144 _perpSr.mintTo(feeCollector, srAumFeeAmt);
```

At `srFeePerc = 0.99e18` that is 99× the outstanding supply, minted in a single call. Two further properties make it sharper than a normal fee knob:

- It is charged *per `rebalance()` call*, not per unit of elapsed time. Nothing in `_chargeAUMFees` reads a timestamp. Shortening `rebalanceFreqSec` through `updateRebalanceWindow` — also owner-only, also unbounded below except for being non-zero — multiplies the accrual rate directly.
- `rebalance()` is permissionless, so the owner does not even need to send the transaction that harvests the fee.

Preconditions. A malicious or compromised owner key. No market condition, no other actor.

Attack sequence. `updateProtocolFeeConfig(0, attacker)` → `updateAumFeePerc(0.99e18, 0.99e18)` → wait for the next rebalance window (at most `rebalanceFreqSec`, which the same owner can shorten) → anyone calls `rebalance()` → the collector holds 99% of both notes → redeem.

Impact. Effectively the entire value of both notes, taken without an upgrade and without touching `PerpTranche.updateController`. It is the cheapest route from a compromised owner key to protocol funds, and unlike an upgrade it leaves the contracts looking untouched.

Proof of concept. proofs-of-concept/core/MED-05-aum-fee-dilution.ts:

```
perpSr supply 50000.0 -> 5000000.0
holder's ownership 100.0% -> 1.0%
fee collector now holds 4950000.0 perpSr and 9900000.0 perpJr
updateAumFeePerc(1e18 - 1) accepted; only exactly 1e18 is rejected
```

Recommendation.

1. Bound the setter to a value that can only ever be a fee: `perpSrAUMFeePerc_ <= MAX_AUM_FEE_PERC` with the maximum a compile-time constant on the order of 10^{16} (1%).
2. Make the charge time-weighted rather than call-weighted: $f_{\text{effective}} = f_{\text{annual}} \times \Delta t / 365\,\text{days}$, with Δt taken from `lastRebalanceTimestampSec`. This also removes the incentive to shorten the rebalance cadence for revenue.
3. Put the setter behind the same timelock as the upgrade path.

```
uint64 internal constant MAX_AUM_FEE_PERC = 1e16; // 1% per year

function updateAumFeePerc(uint64 sr_, uint64 jr_) external onlyOwner {
  if (sr_ > MAX_AUM_FEE_PERC || jr_ > MAX_AUM_FEE_PERC) revert InvalidPerc();
  ...
}

// PerpController._chargeAUMFees
uint256 dt = block.timestamp - lastRebalanceTimestampSec;
uint256 srFeePerc = feePolicy.perpSrAUMFeePerc().mulDiv(dt, 365 days);
```

Regression test. Assert that `updateAumFeePerc` rejects anything above the cap, and that calling `rebalance()` n times over a fixed interval mints the same total as calling it once over that interval, to within rounding.

7.5 LOW-01 — vmDR is not clipped, and the fee curve overflows on it

Severity	Low
Confidence	High — arithmetic confirmed by proof-of-concept; reachability remote
Status	Open

Affected core-contracts/contracts/PerpController.sol computeDRs (L919–L934); core-contracts/contracts/_utils/LineHelpers.sol avgY (L29–L48), computePiecewiseAvgY (L63–L126); core-contracts/contracts/_utils/PerpHelpers.sol volatilityMultiplier (L216–L229).

Category Arithmetic overflow; unhandled revert path.

Description. computeDRs clips one of its two outputs and not the other:

```
924 subDR_ =
925   (s.perpSr.reserveValue > 0)
926     ? s.perpJr.reserveValue
927         .mulDiv(ONE, s.perpSr.reserveValue)
928         .mulDiv(ONE, targetSystemRatio_)
929         .clip(0, INF_DEVIATION_RATIO)
930     : INF_DEVIATION_RATIO;
931 vmDR_ = s.perpJr.vm.mulDiv(ONE, targetPerpJrVM_);
```

The clip on subDR is load-bearing: LineHelpers.computePiecewiseAvgY multiplies a segment's average y by that segment's width, and both grow linearly in the ratio, so the product grows quadratically. At the INF_DEVIATION_RATIO ceiling ($2^{120} - 1 \approx 1.33 \times 10^{36}$) the product peaks around 1.05×10^{72}, comfortably inside int256. Without a clip it does not stay there.

vmDR can reach $2^{128} - 1$ scaled by ONE/targetPerpJrVM. PerpHelpers.volatilityMultiplier blends per-tranche multipliers by value, and BondsManager.trancheData assigns INF_VOL_MULTIPLIER ($2^{128} - 1$) to a junior tranche whose valueRatio truncates to zero. With the initialization default targetPerpJrVM = 1.33e18 that is $\approx 2.56 \times 10^{38}$ — about 192× the protocol's own ceiling — and the fee curve's weighted product overflows int256 at any vmDR above $\approx 1.35 \times 10^{38}$. The operation reverts with Panic(0x11) instead of the intended OpDisabled.

Reachability — stated plainly. This needs the saturated tranche to carry more than about 53% of PerpJr's reserve value. valueRatio truncates to zero only while jrClaim/totalClaim $< 10^{-18}$, and the tranche's reserve value in that band is a few wei, so PerpJr must hold essentially nothing else. That is a PerpJr that has been reduced to dust. The proof-of-concept constructs the state step by step, including finding a settlement price that leaves a junior claim of 499,999 wei against 10^{24} of collateral, but it took a solved-for price to land inside a window roughly 10^{-18} wide. This is reported as Low for that reason, not because the arithmetic is in doubt.

Proof of concept. proofs-of-concept/core/MED-01-vmDR-overflow.ts, six steps:

```
trancheClaim=499999 totalClaim=1e24 valueRatio=0 vm=340282366920938463463374607431768211455
value(jrSupply) = 499999 wei
blended vm = 340282366920938463463374607431768211455   tvl = 5
subDR = 132922799578491587290380706028034457 5  (== the ceiling, clipped)
vmDR  = 25585140369995373192734932889606632440 2  -> 192x the same ceiling
FeePolicy.computeOpFees reverts with Panic(0x11)
the same call with vmDR clipped to INF_DEVIATION_RATIO returns totalFeePerc = 1.015e18
overflow threshold vmDRPost = 135214501657638814832126397408061836362
```

Recommendation. One line, symmetric with the sibling ratio:

```
vmDR_ = s.perpJr.vm.mulDiv(ONE, targetPerpJrVM_).clip(0, INF_DEVIATION_RATIO);
```

Separately, consider bounding `targetPerpJrVM` below as well as above in `updateDRTargets`; today it may be set to one wei, which scales `vmDR` by 10^{18} and reaches the overflow band from an ordinary multiplier.

Regression test. Feed `computeOpFees` the full representable range of both ratios and assert it never panics: it either returns, or reverts with a named error.

7.6 LOW-02 — `deposit` takes collateral and mints nothing below one unit of value

Severity	Low
Confidence	High — confirmed by proof-of-concept
Status	Open

Affected `core-contracts/contracts/bonds/BondsManager.sol` `deposit` (L262–L296).

Description. `deposit` converts collateral to bond value before splitting it:

```
281 uint256 valueIn = underlyingAmt.mulDiv(bd.openPrice, ONE);
282 srTrAmt = TokenAmount(bd.senior, valueIn.mulDiv(bd.srTrancheRatio, ONE));
283 jrTrAmt = TokenAmount(bd.junior, valueIn - srTrAmt.amount);
284
285 bd.underlying.safeTransferFrom(msg.sender, address(this), underlyingAmt);
286 bd.underlyingBal += underlyingAmt;
```

The only zero-check is on `underlyingAmt`, not on `valueIn`. When underlyingAmt $\times$ openPrice $< 10^{18}$ both tranche amounts floor to zero, yet the transfer still executes and `underlyingBal` still increases. The depositor receives nothing and the amount is donated to the bond's existing tranche holders.

Preconditions. A low-priced underlying. The threshold is 10^{18}/openPrice wei, so at an open price of 10^{12} ($\$10^{-6}$ per token) any deposit below 10^{6} wei is silently consumed. At BTC-scale prices the threshold is sub-wei and the issue is unreachable.

Impact. Dust loss for the depositor. Reported because it is a silent success on a failed operation: an integrator batching many small deposits, or a UI that does not check the returned amounts, loses the difference without any signal.

Proof of concept. `proofs-of-concept/core/LOW-bonds-and-controller.ts`, first case: `deposited 999999 wei -> srMinted=0 jrMinted=0`, with the bond's `underlyingBal` up by exactly that amount.

Recommendation.

```
uint256 valueIn = underlyingAmt.mulDiv(bd.openPrice, ONE);
if (valueIn == 0) revert UnexpectedAmount();
```

7.7 LOW-03 — `redeemImmature` burns senior without the paired junior

Severity	Low
Confidence	High — confirmed by proof-of-concept
Status	Open

Affected `core-contracts/contracts/bonds/BondsManager.sol` `redeemImmature` (L456–L513).

Description. Pre-maturity redemption is meant to burn both legs in the ratio of their supplies. The junior leg is computed with a floor division:

```
482 uint256 srAmtRedeemed = srAmtAvail;
483 uint256 jrAmtRedeemed = jrSupply.mulDiv(srAmtAvail, srSupply);
484 if (jrAmtRedeemed > jrAmtAvail) {
485   srAmtRedeemed = srSupply.mulDiv(jrAmtAvail, jrSupply);
486   jrAmtRedeemed = jrAmtAvail;
487 }
488 underlyingAmtReturned = bd.underlyingBal.mulDiv(srAmtRedeemed, srSupply);
```

When srAmtAvail × jrSupply < srSupply the junior amount floors to zero while the senior amount and the underlying return do not. The caller burns senior tranches, burns no junior tranches, and takes collateral out, so the junior's claim per unit strictly falls and the senior's strictly rises.

Impact. Dust. The condition bounds `srAmtAvail` below srSupply/jrSupply, which for a 90/10 bond is 9 wei, so at most a few wei of value moves per call and gas dominates by many orders of magnitude. Reported because it is a genuine break of the stated proportionality property rather than because it is profitable, and because it also means the two supplies can drift out of their minted ratio over time.

Proof of concept. `proofs-of-concept/core/LOW-bonds-and-controller.ts`, second case, on a 90/10 bond: `srBurnt=5 jrBurnt=0 underlyingOut=5`, and the junior claim per unit moves from `1000000000000000000` to `999999999999999990`.

Note on the perps. The notes reach this function through `TrancheHelpers.redeemMax`, which passes the note's whole balance of both legs. A note holding only senior tranches passes `jrAmtAvail = 0` and returns early, so the protocol's own reserves do not trigger this path.

Recommendation. Round the junior leg up, or refuse the redemption when it would round to zero:

```
uint256 jrAmtRedeemed = jrSupply.mulDiv(srAmtAvail, srSupply, Math.Rounding.Ceil);
```

Rounding up keeps the burn proportional or slightly senior-unfavourable, which is the correct direction for a function whose job is to unwind a pair.

7.8 LOW-04 — The controller trusts the fee policy's protocol-share ordering

Severity	Low
Confidence	High — confirmed by proof-of-concept
Status	Open

Affected `core-contracts/contracts/PerpController.sol` `_computeOpFees` (L1288–L1304), `_computeFeeAmts` (L1383–L1389), `collectedFeeBal` (L985–L987), `_settleFees` (L1084–L1131).

Description. `_computeFeeAmts` derives two independent amounts from the policy's two percentages and books them separately: `totalFeeAmt` is what the controller actually withholds, `protocolFeeAmt` is what it earmarks in `protocolFeeBals`. Nothing checks that the second is no larger than the first. `collectedFeeBal` is then a bare subtraction of the earmark from the balance:

```
985 function collectedFeeBal(IERC20 token) public view returns (uint256) {
986   return (token.balanceOf(address(this)) - protocolFeeBals[token]);
987 }
```

An over-reporting policy makes the earmark exceed the balance permanently. Every consumer of `collectedFeeBal` then reverts with `Panic(0x11)`: fee settlement, and with it `rebalance()`, `mint2` on the junior side and `flashMint2` on the junior side. The fee collector cannot repair it either, because `pullProtocolFees` decrements the earmark and transfers the same amount out, preserving the gap.

The canonical `FeePolicy` does clamp with `Math.min(f.totalFeePerc, protocolMaxOpFeePerc)`, so this is only reachable through a non-canonical policy. Worth noting for the record: an explicit guard for exactly this condition was added and then removed again inside the reviewed commit, on the reasoning that the policy is owner-installed.

Impact. Permanent denial of service on fee settlement and rebalancing, requiring an upgrade to clear. The trigger is an owner action, but an *honest* one. The failure mode of installing a slightly wrong policy is not "fees are wrong", it is "the protocol stops rebalancing forever".

Proof of concept. `proofs-of-concept/core/LOW-bonds-and-controller.ts`, third case: `controller holds 10000000000000000000 perpSr, earmarked 50000000000000000000`, and `collectedFeeBal` reverts with `Panic(0x11)` thereafter.

Recommendation. Restore the guard. It is two lines next to one that already exists, it costs one comparison per operation, and the failure it prevents is unrecoverable:

```
if (fees.totalFeePerc >= ONE) revert OpDisabled();
if (fees.protocolFeePerc > fees.totalFeePerc) revert InvalidPerc();
```

Alternatively make `collectedFeeBal` saturating (`clipDiff`), which turns an unrecoverable brick into a recoverable mis-accounting. The guard is better, because it fails the operation instead of silently mispricing every holder's share.

7.9 LOW-05 — `upkeep()` reverts `NoOp`, so the routers' upkeep flags are unusable

Severity	Low
Confidence	High — confirmed by proof-of-concept
Status	Open

Affected `core-contracts/contracts/PerpController.sol` `upkeep` (L465–L477); `core-contracts/contracts/Router.sol` `trancheAndMint` (L71–L147); `vaults/contracts/routers/UsdPerpSrVaultRouter.sol` `redeemUsd` (L121–L160).

Description. `upkeep()` reverts when it finds nothing to do:

```
472 bool j1 = _recoverTranches();
473 (bool j2, ) = _issueAndRegisterTranches();
474 if (!j1 && !j2) {
475   revert NoOp();
476 }
```

That is reasonable for a keeper, which wants a signal. It is not reasonable for the two router entry points that call it as an optional preparatory step. `Router.trancheAndMint(..., doUpkeep = true)` and `UsdPerpSrVaultRouter.redeemUsd(..., execUpkeep = true)` both call it unconditionally and inherit the revert, so a user who sets the flag is refused service whenever the system is already up to date — the steady state. The same call with the flag cleared succeeds.

It is also grief-able where the state *is* dirty: an attacker who front-runs with a bare `upkeep()` (or, for the vault, a bare `vault.upkeep()`) turns the victim's flagged call into a revert for the cost of one transaction.

Proof of concept. `proofs-of-concept/core/MED-03-op-disabled-freeze.ts`, last case: `doUpkeep=true reverted NoOp at rest; doUpkeep=false succeeded.`

Recommendation. Make the optional step tolerant at the call site, which keeps the keeper's signal intact:

```
if (doUpkeep) {
  try controller.upkeep() {} catch {}
}
```

or expose a non-reverting `tryUpkeep() returns (bool)` on the controller and have the routers call that.

7.10 LOW-06 — The vault charges depositors for resting underlying but never pays redeemers for it

Severity	Low
Confidence	High — confirmed by proof-of-concept
Status	Open

Affected `vaults/contracts/UsdPerpSrVault.sol` `computeMintAmt` (L955–L998), `computeRedemptionAmts` (L1004–L1011), `getTVL` (L1015–L1021), `_swapSellUnderlying` (L1219–L1242).

Description. The passive sell leg sizes its flash redemption for the worst fee it would accept and pays out the exact amount promised, so a cheaper realised fee leaves the difference behind as a resting `underlying` balance. `getTVL()` deliberately excludes it, and the two user-facing functions treat it asymmetrically:

```
uint256 underlyingBal = underlyingBalance();
if (underlyingBal > 0) {
  uint256 restingVal = _underlyingToUsdVal(underlyingBal, _underlyingOraclePrice());
  uint256 tvl = getTVL();
  notesMinted = notesMinted.mulDiv(tvl, tvl + restingVal);   // depositor pays
}
```

```
usdAmtOut = usdBalance().mulDiv(notesAmt, totalSupply_);
perpSrAmtOut = perpSrBalance().mulDiv(notesAmt, totalSupply_);  // redeemer is not paid
```

A depositor is charged their pro-rata share of the resting balance and a redeemer forfeits theirs. The difference accrues to whoever stays. A deposit followed immediately by a redemption loses $A \cdot R/(\mathrm{TVL} + R + A)$.

The repository's own suite pins the conservation property on the deposit side ("a deposit never lowers the value behind an existing note"). The redemption side has no corresponding assertion, which is why this survived.

Impact. Bounded by the resting balance, which is bounded by `maxFlashRedeemFeePerc` of the sell leg's size — 10 bps by default. But `upkeep()` refuses to clear a resting balance worth less than `minOpUsdVal`, so a sub-threshold remainder persists indefinitely and taxes every redemption until a later trade grows it past the floor.

Proof of concept. `proofs-of-concept/vaults/LOW-vault-findings.ts`. With a $150,000 vault holding $1,000 of resting underlying, a $15,000 round trip loses $90.36 — exactly $AR/(\mathrm{TVL} + R + A)$ — and a quarter-of-supply redemption forfeits $250 of underlying:

```
round trip: usd -60240964 (6dp), perpSr -30120481927710843374 (18dp)
            -> net -90.361445927710843374 USD of value
redeem 25000.0 notes: usdOut=25000000000 perpOut=12500000000000000000000
            forfeited underlying = 250.0
```

Recommendation. Make the two sides symmetric. Either return the redeemer's share of the resting balance:

```
underlyingAmtOut = underlyingBalance().mulDiv(notesAmt, totalSupply_);
```

(and transfer it in `redeem`, adding a third amount to the `Redeem` event), or include the resting balance in `getTVL()` and drop the deposit-side haircut. The first is the smaller change and keeps the "`getTVL` is the tradeable balance sheet" reading intact.

Regression test. Mirror the existing deposit-side conservation test: assert that a redemption never raises the value behind a remaining note, measured on the whole balance sheet including resting underlying, at 1×, 400× and 40000× the resting balance.

7.11 LOW-07 — `updateTargetDR` accepts a zero lower bound

Severity	Low
Confidence	High — confirmed by proof-of-concept
Status	Open

Affected `vaults/contracts/UnderlyingPerpVault.sol` `updateTargetDR` (L254–L264); `vaults/contracts/UsdPerpSrVault.sol` `updateTargetDR` (L391–L401); `vaults/contracts/_utils/PerpVaultHelpers.sol` `isAtEquilibrium` (L48–L53), `computeRebalanceSize` (L151–L172).

Description. The equilibrium band added in the reviewed commit is validated only for bracketing its target:

```
254 function updateTargetDR(uint96 targetDR_, DRRange memory eqDRRange_)
255   public onlyOwner
256 {
257   if (targetDR_ <= 0) revert InvalidParamDefn();
258   if (eqDRRange_.lower > targetDR_ || eqDRRange_.upper < targetDR_) {
259     revert InvalidRange();
260   }
261   ...
262 }
```

`lower = 0` passes. `computeRebalanceSize` consults `isAtEquilibrium` before anything else and returns a zero amount for the whole half-line below the target, so a senior-leg vault holding that band will size no trade at any deviation below its target — including `subDR = 0`, which is what a total junior collapse looks like and precisely the moment a senior vault most wants to de-risk. The gate lives in the shared helper, so previews, `maxAcceptableTerms`, `upkeep` and both entry points all inherit it consistently; there is no separate path that still trades.

The commit message for the band records this behaviour explicitly, so this is a deliberate degree of freedom rather than a missed check. It is reported because the setter gives an operator no signal that a one-sided band is different in kind from a symmetric one, and because the resulting failure is silent — the vault simply reverts `NoOp` forever.

Proof of concept. `proofs-of-concept/vaults/LOW-vault-findings.ts`, third case: with `eqDRRange = [0, 1e18]` and the junior fully impaired, `subDR = 0.0`, `rebalanceCapacity()` returns amount 0, and `rebalance()` reverts `NoOp`.

Recommendation. Require the band to be a band: `eqDRRange_.lower > 0`, and optionally cap its width as a fraction of `targetDR_` so an operator cannot install one wide enough to disable the vault. Whatever the code decides, the deployment modules should carry a comment at each configured band saying what deviation the vault will and will not act on.

7.12 LOW-08 — Stateless routers grant unbounded approvals to caller-supplied addresses

Severity	Low
Confidence	Medium — source-confirmed; not reduced to a working theft
Status	Acknowledged

Affected `core-contracts/contracts/Router.sol` `_ensureApproval` (L462–L470) and every entry point; `vaults/contracts/_utils/RouterHelpers.sol` `ensureApproval`, `ensureInfiniteApproval`, `forwardERC20` (L19–L45); `vaults/contracts/routers/PerpVaultRouter.sol` (both functions).

Description. The routers are designed to hold nothing between transactions and to sweep whatever they hold to `msg.sender` at the end of a call. To make that work they grant `type(uint256).max` allowances lazily. In `PerpVaultRouter` every address involved — the tokens, the controller, the DEX strategy — is a function parameter:

```
52 usd_.safeTransferFrom(msg.sender, address(this), usdAmtIn);
53 usd_.ensureApproval(address(dexStrategy_), usdAmtIn);
54 underlying_.ensureInfiniteApproval(address(perpController_));
55 ...
56 perp_.safeTransfer(msg.sender, perpAmtOut);
```

So anyone may cause the router to hold an unlimited standing allowance on an arbitrary token to an arbitrary contract, at the cost of supplying one wei of that token. Combined with the full-balance sweep, any token that ever comes to rest in a router — an accidental transfer, an integration that over-delivers, a token with unusual accounting — is claimable.

The vault routers state this as an invariant in their own NatSpec ("holds no balances between txs; any tokens or standing allowance left here are sweepable by the next caller. Not custody"), and the vault suite already contains a test named *"reentry on transfer-callback USD allows attacker to steal mid-call dust"*. It is recorded here as *Acknowledged* for that reason. The core `Router` carries no equivalent warning.

Impact. Bounded by whatever is stranded, which by design is nothing. The risk is that the design invariant is stronger than it looks: it holds only as long as every integration returns exactly what it delivers.

Recommendation. Approve exactly what the call needs and reset to zero on the way out, rather than approving `max` and leaving it. Where the allowance genuinely must be infinite, restrict the spender to an immutable set fixed at construction rather than a parameter. At minimum, replicate the vault routers' warning on `Router` so integrators see it.

7.13 LOW-09 — `mature()` is permissionless with no deadline, so the caller picks the settlement price

Severity	Low
Confidence	Medium — source-confirmed; economic magnitude not measured
Status	Open

Affected `core-contracts/contracts/bonds/BondsManager.sol` `mature` (L310–L331), `computeSettlement` (L703–L718).

Description. Settlement takes whatever the oracle says at the moment somebody calls it:

```
310 function mature(bytes32 bondID) external override
311   onlyValidBond(bondID) onlyMaturable(bondID) nonReentrant
312 {
313   BondData storage bd = bondsData[bondID];
314   uint256 closePrice = assetPrice(bd.underlying);
315   (bd.srClaim, bd.jrClaim) = computeSettlement(
316     bd.srTrancheRatio, bd.underlyingBal, bd.openPrice, closePrice
317   );
318   ...
319 }
```

`onlyMaturable` requires only that the maturity date has passed and the bond is not already settled. There is no upper bound on the delay and no restriction on the caller. Whoever calls first freezes the senior/junior split at the price of that block, and the split is a strictly decreasing function of the close price on the senior side.

Before settlement the bond keeps re-pricing against the live oracle, so calling `mature()` produces no instantaneous jump — an attacker cannot mint, settle and redeem for a profit in one transaction. What they gain is the option to *lock in* a level. A holder of PerpJr who calls at a local price maximum freezes a high junior claim that no subsequent fall can undo, and vice versa for PerpSr. The counterparty for that transfer is the other note's holders.

Preconditions. A gap between the maturity timestamp and the first `mature()` call during which the price moves. In practice that gap is whatever the keeper's cadence is; `upkeep()` and `rollover()` both call `cleanupMature()`, and both are permissionless, so an attentive market settles bonds promptly. The exposure is therefore a keeper-liveness exposure.

Impact. A value transfer between PerpSr and PerpJr holders proportional to the price move over the settlement delay, scaled by the affected bond's share of each note's reserves. Not a theft from the protocol; a redistribution between its two sides.

Recommendation.

1. Prefer a price anchored to the maturity timestamp rather than to the calling block — for a Chainlink source, resolve the round that was current at `maturityDate` via `getRoundData` and a caller-supplied round hint, validated against `maturityDate`.
2. Failing that, bound the drift: reject `mature()` if `block.timestamp - maturityDate` exceeds a settlement window, and have the keeper's alerting treat a bond entering that window as a page.

3. Whichever is chosen, state in user documentation that settlement price is “price at first settlement call after maturity”, because that is what it is today.

7.14 LOW-10 — `updateBondConfig` under-validates and can stop issuance

Severity	Low
Confidence	High — source-confirmed
Status	Open

Affected `core-contracts/contracts/strategies/BondIssuer.sol` `updateBondConfig` (L82–L91), `issue` (L115–L164); `core-contracts/contracts/bonds/BondsManager.sol` `createBond` (L199–L259).

Description. `updateBondConfig` checks only the upper bound of the tranche ratio:

```
82 function updateBondConfig(uint64 srTrancheRatio_, uint32 maxMaturitySec_)
83   external onlyOwner
84 {
85   if (srTrancheRatio_ >= ONE) {
86     revert InvalidPerc();
87   }
88   srTrancheRatio = srTrancheRatio_;
89   maxMaturitySec = maxMaturitySec_;
90 }
```

`srTrancheRatio_ = 0` is accepted here and rejected downstream by `createBond`'s `srTrancheRatio_ <= 0` guard, so `issue()` reverts instead of returning `(false, ...)`. `maxMaturitySec_` is not validated at all; if it is smaller than the elapsed time into the current issuance window, `nextMaturityDate` lands at or before `block.timestamp` and `createBond`'s maturity guard reverts for the same reason.

That matters more than a normal misconfiguration because `issue()` is not called in isolation. `PerpController._issueAndRegisterTranches` calls it from `mint2`, `flashMint2`, `rollover` and `upkeep`, none of which catch the revert. A single bad value therefore stops minting and system maintenance, not just issuance.

Impact. Owner-triggered denial of service across most of the protocol, recoverable by the owner setting a valid value again.

Recommendation. Validate at the setter, where the operator gets the error:

```
if (srTrancheRatio_ == 0 || srTrancheRatio_ >= ONE) revert InvalidPerc();
if (maxMaturitySec_ <= minIssueTimeIntervalSec) revert InvalidParamDefn();
```

The second condition also keeps the active-bond set bounded, since its size is approximately maxMaturitySec/minIssueTimeIntervalSec and `cleanupMature()` iterates it.

7.15 LOW-11 — Rollover divides by `valueRatio` with no zero guard

Severity	Low
Confidence	Medium — source-confirmed; trigger state not constructed
Status	Open

Affected `core-contracts/contracts/strategies/PerpBalancer.sol` `computeRolloverPolicy` (L426–L430), `computeBackupRolloverPolicy` (L533–L536).

Description. Both rollover policies invert a senior tranche's `valueRatio` to size the deposit that will reproduce a given value:

```
426 uint256 underlyingAmtIntoBond = rolledValPerFreshTranche[i].mulDiv(
427   ONE, srTrData.data.valueRatio, Math.Rounding.Ceil
428 );
```

`valueRatio` is a `uint64` produced by `trancheClaim.mulDiv(ONE, totalClaim)` and can truncate to zero for a senior tranche whose claim has collapsed relative to its bond — which happens when the close price rises far enough that srRatio × openPrice/closePrice $< 10^{-18}$. The division then panics and the rollover reverts.

Reachability. Fresh tranches are created at the current price, so their ratio starts near `srRatio`; reaching a 10^{18}-fold price appreciation before a tranche becomes stale is not a realistic market path for the assets this protocol targets. This is reported as a missing guard on a divisor that the type system does not bound, not as a live risk. It is the same class of omission as LOW-01 — an unbounded intermediate reaching arithmetic that assumes it is bounded.

Recommendation. Skip a fresh tranche whose `valueRatio` is zero rather than dividing by it, so one degenerate bond cannot stop the rollover for all the others:

```
if (srTrData.data.valueRatio == 0) continue;
```

and in `computeBackupRolloverPolicy`, exclude zero-ratio candidates when selecting the preferred bond rather than after selecting it.

8 Informational observations

These are hardening, maintainability and monitoring notes. None has a credible loss scenario at the reviewed commit.

INFO-01 — Fee settlement moves underlying with no `ReserveSynced`

`core-contracts/contracts/PerpController.sol` L1126–L1128. Every other reserve movement calls `logBalance`, which emits `ReserveSynced`; this one does not:

```
1126 if (collectedUnderlyingBal > 0) {
1127   underlying.safeTransfer(address(_perpJr), collectedUnderlyingBal);
1128 }
```

`ReserveSynced` is exactly the event the subgraph indexes PerpJr's underlying balance from (`subgraph/src/perp-token.ts` L122–L143, `subgraph/subgraph.yaml` L112–L113), so the indexed balance drifts from the chain on every settlement. Confirmed by proof-of-concept: PerpJr's balance moves from 0 to 10^{18} with zero `ReserveSynced` events for the underlying. Fix: call `_perpJr.logBalance(underlying)` after the transfer, matching the pattern used everywhere else.

INFO-02 — Unregistered tranches sent to a note are permanently stranded

`core-contracts/contracts/perps/PerpTranche.sol` `_syncTranche` (L300–L312) only ever runs for tranches that appear in a controller-driven transfer, and `registerTranche` only ever adds the current bond's legs. A tranche token transferred directly to a note while it is not in the registered set is invisible to `reserveData()`, is never valued, and is never redeemed by `recover()`. It cannot be recovered without an upgrade. Consider a permissionless `sync(ITranche)` that validates the token through `IBondsManager.isValidTranche` and then calls `_syncTranche`.

INFO-03 — `computeMintAmt` divides by zero when both vault legs are empty

`vaults/contracts/UnderlyingPerpVault.sol` L483–L491. The two single-asset branches are guarded; the fall-through is not:

```
483 } else {
484   underlyingAmtIn = underlyingAmtMax;
485   perpAmtIn = perpBal.mulDiv(underlyingAmtIn, underlyingBal);  // underlyingBal may be 0
```

Reaching it requires both balances at zero with supply non-zero, which `MINIMUM_LIQUIDITY` makes hard but does not make impossible under repeated floor-rounding withdrawals. A donation of either asset revives the vault, so the consequence is a temporary revert rather than a permanent brick. Add an explicit guard returning `(0, 0, 0)`.

INFO-04 — `cachePrice()` and `resetCache()` are permissionless

`core-contracts/contracts/oracles/CachedOracle.sol` L41–L50. Anyone can seed or clear the transient cache. This is not exploitable against a per-block price source — `cachePrice` re-fetches and re-validates, so a pre-seeded value equals the value a fresh fetch would return in the same block — but it does mean `getValidPrice()` can return a value that skipped validation this call, and it widens the surface if a future oracle implementation is ever intra-block variable. Restricting both to a small allowlist, or making `resetCache` idempotent-only-for-the-setter, costs nothing.

INFO-05 — `answeredInRound < roundId` is a deprecated check

`core-contracts/contracts/oracles/CLOracleWrapper.sol` L58. `answeredInRound` has been deprecated by Chainlink and is not populated meaningfully by newer aggregator versions; the check is harmless but conveys no information. The `updatedAt`-based staleness check on the preceding line is the one doing the work.

INFO-06 — Reserve reads scale with tranche count and repeat per operation

`PerpTranche.maxTrancheCount()` is 50. `reserveData()` makes one `data()` call per registered tranche, and each of those reaches `BondsManager.trancheData` which calls `assetPrice` — an external oracle read. `PerpController.mint2` can call `systemReserveState()` twice, and `flashMint2` up to three times, each touching both notes. In the expected configuration the tranche count is maxMaturitySec/minIssueTimeIntervalSec (four with the defaults), so this is comfortable today; it is worth a gas assertion in CI so a future configuration change does not walk into a block-limit problem quietly.

INFO-07 — `Router.redeem3` sweeps the router's whole balance

`core-contracts/contracts/Router.sol` L203–L226 pays out `underlying.balanceOf(address(this))` and each tranche's full balance rather than the amounts the operation produced. This is the same design the vault routers document explicitly and the core router does not. See LOW-08.

INFO-08 — `_enforceMintCap` ignores `vmDR` and rejects everything at `subDR == 1`

`core-contracts/contracts/PerpController.sol` L1350–L1356:

```
1350 bool movesToTarget =
1351   sd.subDRPre < ONE
1352     ? (sd.subDRPost > sd.subDRPre && sd.subDRPost <= ONE)
1353     : (sd.subDRPost < sd.subDRPre && sd.subDRPost >= ONE);
```

At `subDRPre == ONE` the else branch requires `subDRPost` to be both strictly below and at least `ONE`, so no single-sided mint can ever pass the cap at exactly the target. That is conservative rather than wrong, but it is a discontinuity at the most likely operating point and worth an explicit `==` case. Separately, the cap exemption reasons only about `subDR`; a mint that improves `vmDR` while leaving `subDR` flat is rejected.

INFO-09 — No privileged setter emits an event

Across both packages, not one `update*` / `add*` / `remove*` / `pause*` function emits an event. `FeePolicy` and `BondIssuer` declare no events at all. The system has more than twenty privileged capabilities (§10); today none of them is observable off-chain except by diffing storage. Add an event per setter carrying the old and new value. This is the single cheapest improvement to the protocol's operational security posture, because it is what makes a compromised-key incident detectable in minutes rather than at the next audit.

INFO-10 — Address setters accept the zero address

These setters all accept `address(0)` without complaint:

- `PerpController`: `updateKeeper`, `updateFeePolicy`, `updatePerpBalancer`, `updateIssuer`

- `FeePolicy`: `updateProtocolFeeConfig`
- `BondsManager`: `updateOracle`
- both vaults: `updateKeeper`, `updateSwapper`

Some of those are recoverable (`updateKeeper`), some brick an operation until the owner corrects them (`updateFeePolicy`), and `BondsManager.updateOracle(asset, 0)` makes `assetPrice` revert `UndefinedOracle` for that asset, which is the same protocol-wide freeze as MED-03. Add zero-address guards, or at minimum on the three that can freeze the system.

9 Economic and design risks

These are not implementation defects. They are properties of the design that can still cost users money, and they are listed separately so they are not lost among the code findings.

9.1 The rebalance is a scheduled, predictable value transfer

`rebalance()` moves up to `tvlPercLimits.upper` (2.5% by default) of the payer note's TVL to the other note, once per window, and it is permissionless. Its direction and size are computable from public state before it happens. The generic attack is to flash-mint the receiving note immediately before, call `rebalance()`, and flash-redeem immediately after — all three in one transaction, because `ReentrancyGuardTransient` releases per external call, not per transaction.

With the shipped fee curves this is unprofitable by a wide margin. At `subDR` near 1.0 the round trip costs roughly 1% on the mint (the `subDRUp` curve) plus roughly 2.9% on the redemption (the average of `subDRDown` over the same interval) plus two 0.5% flash base fees, against a capture of at most $\approx 0.83\%$ of the position. The margin is comfortable but it is *entirely* a function of the fee parameters: the vaults' own defaults anticipate flash fees around 10 bps (`maxFlashMintFeePerc` and `maxFlashRedeemFeePerc` are both `0.001e18`), and at that level the arithmetic inverts. Any future reduction in the impact curves must be checked against this specific sequence before it ships.

9.2 MEV on the vault rebalance is bounded by `oracleDeviationPerc`

Both vault rebalance legs price against the underlying oracle and enforce a proportional rate floor, so a searcher who moves the DEX pool can extract at most the configured deviation — 15 bps by default — of whatever notional the vault trades, and only up to `maxDeploymentVelocityPerc` (33%) of TVL per session. That is a deliberate, priced cost of rebalancing rather than a leak, but at the default settings the ceiling is roughly $0.33 \times 0.0015 \approx 5$ bps of TVL per session, which is meaningful when compounded daily. The `whenUnderlyingHealthy` cross-check (250 bps against the oracle) is a sanity gate, not the pricing gate; the pricing gate is `oracleDeviationPerc`, and it is the number to tune.

9.3 The senior leg's protection is only as good as the settlement price

The whole product rests on `computeSettlement`: the senior claim is $\min(\text{bal}, \lceil \text{bal} \cdot r \rceil \cdot \text{open/close})$. That is a clean, closed-form rule with ceiling rounding deliberately favouring the senior, and it does what the whitepaper says. But it consumes exactly one price at exactly one moment, chosen by whoever calls `mature()` (LOW-09), and it has no smoothing, no TWAP and no confidence interval. A single anomalous print at settlement is written into the split permanently.

9.4 Junior wipeout is a discontinuity, not a slope

Once `jrClaim` reaches zero the junior tranche's value is zero and its volatility multiplier saturates. Approaching that point, `vmDR` grows without bound (LOW-01), the `vmDRUp` fee curve saturates, and mint operations on the junior side start reverting `OpDisabled` — which is to say, the system stops accepting the recapitalisation it needs. The design intent is presumably that junior mints become cheap as `vmDR` falls, but `vmDR` does not fall in a wipeout, it rises. This interaction between MED-01 and the junior's own dynamics deserves an explicit simulation before mainnet.

9.5 Dependency on external venues

The USD vault's active rebalance depends on a Curve two-hop route or a Uniswap v4 pool remaining liquid and un-depegged. `isHealthy()` handles the honest failure modes (EMA window shortened, EMA-vs-spot divergence, crvUSD depeg, insufficient v4 liquidity) by disabling deposits and rebalances — but not redemptions, which is the right call. The residual risk is that the vault sits un-rebalanced through a venue outage while its DR gap widens; the passive swap leg and the CoW broker exist precisely to cover that, and both were reviewed and found sound.

9.6 Bad debt is not modelled

There is no bad-debt concept: the junior simply goes to zero and the senior takes the whole collateral. That is a coherent design for a fully-collateralised tranche and avoids a whole class of socialised-loss bugs. Worth stating explicitly because readers coming from lending protocols will look for it.

10 Centralization and privileged operations

Every capability below is exercisable by a single key at the reviewed commit. No timelock, delay, or rate limit is enforced in code anywhere in either package. The prior review recorded that the owner is a documented multisig; this review could not verify that on-chain and treats it as an operational control rather than a code control.

Capability	Role	Maximum impact
Upgrade any proxy	Owner	Total. Arbitrary code over all custodied assets.
`PerpTranche.updateController`	Owner	Total. Points a note at an attacker contract that can mint, burn and move its entire reserve.
`FeePolicy.updateAumFeePerc`	Owner	Near-total without an upgrade — 99% of both notes in one permissionless `rebalance()` (MED-04).
`FeePolicy.updateFeeParams`	Owner	Set any curve, including one that disables all operations (`OpDisabled`) or one that charges 99% on every mint and redeem.
`updateProtocolFeeConfig`	Owner	Redirect the accrued protocol fee share to any address; raise `protocolMaxOpFeePerc` to 100%.
`PerpController.updateFeePolicy` / `updatePerpBalancer` / `updateIssuer`	Owner	Install arbitrary logic in the fee, rebalance-planning and issuance paths. A malicious balancer returns a plan that moves any amount between the notes.
`updateDRTargets`	Owner	Move the system's target ratio and volatility multiplier, which repositions every fee curve relative to the live state — including into a region where redemptions are frozen (MED-01).
`updateRebalanceWindow`	Owner	Shorten the rebalance period, multiplying AUM fee accrual (MED-04).
`updateLiquidityThresholds`	Owner	Set the PerpJr reserved liquidity to 100% of TVL, which blocks rollovers.
`BondsManager.updateOracle`	Owner	Replace any asset's price source with an arbitrary contract; controls every tranche valuation and every settlement.
`addAsset` / `addIssuer`	Owner	List new collateral and new bond issuers.
`BondIssuer.updateBondConfig`	Owner	Change the senior ratio for future bonds; a bad value halts issuance and, with it, minting and maintenance (LOW-10).
`PerpController.pauseUserOps`	Keeper	Indefinite freeze of `mint2`, `redeem2`, `flashMint2`, `flashRedeem2`. Redemptions included.

continued overleaf

Capability	Role	Maximum impact
`PerpController.pauseSystemOps`	Keeper	Indefinite freeze of `rebalance`, `rollover`, `upkeep`.
`updateValueCap`	Keeper	Set the system TVL cap to zero, blocking all double-sided mints.
Vault `pause`	Keeper	Freeze vault deposits, redemptions and rebalances. Redemptions included.
`UsdPerpSrVault.updateSwapper`	Owner	Whitelist an arbitrary counterparty for the passive swap leg. Bounded by the on-chain price floor and session budget, so this is the least dangerous of the whitelists.
`updateDexStrategy`	Owner	Route the vault's active rebalance through arbitrary code, with the vault's USD and underlying max-approved to it. Bounded by the rate floor the vault checks after the trade.
`updateOracleDeviationPerc`	Owner	Widen the price band both vault legs settle at, up to just under 100%.
`updateHealthCheckParams`	Owner	Disable the USD peg check (`address(0)`) and widen the DEX-vs-oracle divergence cap to 100%.
`initDeposit`	Owner	Seed the vault. One-shot; the only capability here that cannot be repeated.
`pullProtocolFees`	Fee collector	Bounded to `protocolFeeBals`. The one privileged capability with a hard cap.

Recommendations

1. Put a timelock in front of the owner on both packages. The three capabilities that most need it are the upgrade path, `PerpTranche.updateController`, and `updateAumFeePerc`.

2. Bound the parameters that have no natural ceiling, in code rather than in process: AUM fee, protocol op fee, oracle deviation, and the fee curve's maximum value (see MED-01 and MED-04).

3. Split pause authority from parameter authority. A keeper that can freeze redemptions indefinitely is a different trust assumption from a keeper that can stop a runaway rebalance, and the two should not share a key.

4. Consider making `pauseUserOps` auto-expiring — a pause that lapses after N hours unless renewed converts an indefinite freeze into a bounded one without weakening incident response.

5. Publish the role addresses, their thresholds, and the timelock delay, and keep them current. Users cannot price a trust assumption they cannot see.

11 Testing results

11.1 Existing suites

Both suites were executed from a clean checkout of the reviewed commit.

Suite	Passing	Failing	Skipped	Wall clock
core-contracts (FUZZ_RUNS=0)	1,638	0	0	52 s
vaults (test/*.ts)	838	0	0	18 s
Total	**2,476**	**0**	**0**	

Two suites were *not* run and their absence is a real gap: core-contracts/test/fuzz/ (disabled by the test script's FUZZ_RUNS=0 PATH_FUZZ_RUNS=0) and vaults/test/fork/ (requires an ALCHEMY_SECRET that was not available).

11.2 Compiler warnings

94 core and 136 vault source units compile with no warning attributable to an in-scope file. The only warnings emitted come from OpenZeppelin 5.6.1: an identifier-deprecation notice for at in EnumerableSet, and the standard EIP-1153 composability notice for TransientSlot. Both are informational and neither is actionable here.

11.3 Added tests

Twenty-three proof-of-concept tests in seven files, written against the repository's own Hardhat harness and fixtures. All pass at the reviewed commit.

File	Covers	Cases
core/MED-01-vmDR-overflow.ts	LOW-01	6
core/MED-02-perp-donation-inflation.ts	MED-02	2
core/MED-03-op-disabled-freeze.ts	MED-01, LOW-05	4
core/MED-04-oracle-dos.ts	MED-03	2
core/MED-05-aum-fee-dilution.ts	MED-04	2
core/LOW-bonds-and-controller.ts	LOW-02, LOW-03, LOW-04, INFO-01	4
vaults/LOW-vault-findings.ts	LOW-06, LOW-07	3
Total		**23**

11.4 Coverage

Re-measured from scratch with solidity-coverage 0.8.17 as part of this engagement, not taken from the repository's committed reports. Both runs used the project's own test selection, with the added proof-of-concept files moved out of the tree, so the figures are directly comparable to what the project's CI produces. They reproduce the committed numbers exactly, which is a small but real verification in its own right.

core-contracts

File	% Stmts	% Branch	% Funcs	% Lines
PerpController.sol	100	90.68	100	100
Router.sol	100	88.89	100	100
_utils/ERC20Helpers.sol	100	100	100	100
_utils/LineHelpers.sol	100	94.44	100	96.3
_utils/MathHelpers.sol	100	100	100	100
_utils/PerpBalancerHelpers.sol	100	100	100	100
_utils/PerpHelpers.sol	100	100	100	100
_utils/Shrink.sol	100	100	100	100
_utils/TrancheHelpers.sol	100	100	100	100
bonds/BondsManager.sol	100	95	100	100
bonds/Tranche.sol	100	100	100	100
oracles/CLOracleWrapper.sol	100	100	100	100
oracles/CachedOracle.sol	100	100	100	100
oracles/L2CLOracleWrapper.sol	100	100	100	100
perps/PerpJr.sol	100	100	100	100
perps/PerpSr.sol	100	100	100	100
perps/PerpTranche.sol	100	95.24	100	98.77
strategies/BondIssuer.sol	100	91.67	100	100
strategies/FeePolicy.sol	100	100	100	100
strategies/PerpBalancer.sol	100	95.65	100	99.39
All files	**100**	**93.92**	**100**	**99.73**

vaults

File	% Stmts	% Branch	% Funcs	% Lines
UnderlyingPerpVault.sol	100	98.11	100	100
UsdPerpSrVault.sol	100	94.17	100	99.65
PerpVaultHelpers.sol	100	100	100	100
RouterHelpers.sol	100	100	100	100
PerpVaultRouter.sol	100	100	100	100
UnderlyingPerpVaultRouter.sol	100	100	100	100
UsdPerpSrVaultCowBroker.sol	100	100	100	100
UsdPerpSrVaultRouter.sol	100	100	100	100
BaseDexStrategy.sol	100	90	100	100
CurveCbBtcUsdcStrategy.sol	100	100	100	100
UniswapV4CbBtcUsdcStrategy.sol	100	95.65	100	100
All files	**100**	**96.75**	**100**	**99.87**

This is genuinely high coverage, and unusually, it is high in the places that matter — the CoW broker and all four routers are at 100% on every metric. It should still be read for what it is: every line was executed, which is not the same as every line having been executed against an adversary. Four of the findings in this report sit on lines the suite already covers.

11.5 Static analysis

solhint 6.2.3, project configuration

Clean. Zero findings on either package.

Slither 0.11.6

Run against both packages with `-exclude-dependencies`. 105 results across 91 core contracts, 194 across 137 vault contracts. Every result was manually triaged; the table below is the disposition, and *no Slither result is reported in this document as a finding without independent confirmation.*

Detector	Count	Disposition	Reasoning
unchecked-transfer	20	Out of scope	All 20 are in vaults/contracts/_test/ mocks. No in-scope file.
arbitrary-send-erc20	3	Out of scope	All 3 in _test/MockCow.sol.
reentrancy-balance / reentrancy-no-eth	24	False positive (mostly)	The flagged entry points carry nonReentrant from ReentrancyGuardTransient, whose transient-storage lock Slither does not model. The exception is the stateless routers, which genuinely have no guard — corroborating LOW-08, which the vault suite also already tests.
weak-prng	4	False positive	block.timestamp % freq is grid alignment for a rebalance window, not randomness.
uninitialized-state	1	False positive	A mapping needs no initialiser.
uninitialized-local	10	False positive	Accumulators relying on the zero default.
unused-return	45	Reviewed, benign	Ignored EnumerableSet booleans (membership is checked separately), ignored logBalance returns, and destructured Chainlink tuples.
divide-before-multiply	1	Benign	One in-scope hit, BaseDexStrategy.maxFillable's binary-search midpoint. One wei of search precision.
incorrect-equality	1	False positive	A boolean equality on a direction flag.
calls-loop	34	Confirmed, informational	Matches INFO-06.
timestamp	28	By design	Window and maturity comparisons.
events-maths / events-access	25	**Confirmed**	Independently verified and reported as INFO-09.
missing-zero-check	16	**Confirmed**	Independently verified and reported as INFO-10.
naming-convention, assembly, pragma, immutable-states, missing-inheritance, low-level-calls, cache-array-length, cyclomatic-complexity, shadowing-local	61	Style	Constant-case immutables, deliberate assembly in Shrink and GPv2Order, and the like.

11.6 Analyses that were attempted and did not complete

- **Symbolic execution.** Not run. Mythril and Halmos were not available in the environment and installing them was not attempted after Slither's own install required working around a source-only dependency.
- **Storage-layout diffing across an upgrade.** Not applicable — nothing is deployed, so there is no prior layout to diff against. The @openzeppelin/hardhat-upgrades plugin performs its own layout validation on every deployProxy call in the existing suite, and that passed.
- **Gas profiling.** REPORT_GAS was not enabled; INFO-06 is based on reading the loops and on one existing test taking 141 seconds to fill the 50-tranche reserve, not on measured gas.

11.7 Meaningful negative results

Attack classes that were specifically tested and did *not* break an invariant. These are worth as much as the findings.

- **Share-price arithmetic.** Mint, redeem, flash mint, flash redeem and fee settlement each preserve reserve-value-per-note exactly, with no rounding residue — verified symbolically and against the existing round-trip tests. Repeated deposit/withdraw cycles generate no profit.
- **Reentrancy.** Every state-mutating entry point on the controller, the bonds manager, the

issuer and both vaults carries a guard. Cross-contract paths were traced: a hostile token reentering during a fee payout or a bond deposit cannot reach a mutating function, because the notes are `onlyController` and the controller is guarded. The perps' `logBalance` being `onlyController` while called internally is correct — `msg.sender` stays the controller through the internal call.

- **Flash-loan sequencing.** `ReentrancyGuardTransient` does release per call rather than per transaction, so mint → rebalance → redeem *is* composable in one transaction. It was analysed and priced (§11 and the economic section) and is unprofitable at the shipped fee parameters.
- **Bond pool solvency.** Collateral is pooled across all bonds in one contract, which is the kind of design that usually hides a shortfall. It reconciles: deposits add, settlement moves `underlyingBal` into `srClaim + jrClaim` with no loss, and both redemption paths pay the last redeemer the exact remaining claim rather than a floor-rounded fraction.
- **The CoW broker.** This was the component most likely to hold a serious bug — a contract that signs orders via ERC-1271 for a solver it does not trust, inside a flash loan, with the pre-hook's arguments entirely solver-controlled. It holds up. The broker keeps no capital, every leg is price-checked against the oracle by the vault rather than by the broker, the exact-quantity leg is pinned to the underlying so a fill cannot strand inventory, and any solver deviation either reverts on the loan repayment or leaves a surplus that is swept to the vault. Both directions were traced through the full settlement stack.
- **Vault deposit manipulation.** `deposit` is ratio-bound and reads no oracle for pricing the pair, so an LP cannot move the vault's asset mix or profit from a stale price. Confirmed by reading and by the existing suite.
- **Rollover value conservation.** The plan sizes each deposit so the senior tokens returned are worth exactly what left, and the ceiling rounding costs PerpJr $O(1)$ wei per fresh tranche per rollover — as the source comment claims.
- **Tranche supply ratio.** Pre-maturity, senior and junior supplies stay in their minted ratio through both deposit and proportional redemption, except for the dust path in LOW-03.
- **ERC-7201 storage.** `PerpTranche`'s namespaced slot is derived correctly and cannot collide with the inherited OpenZeppelin layouts.
- **Implementation-contract takeover.** Every upgradeable implementation calls `_disableInitializers()` in its constructor, so an implementation cannot be initialised directly and used to seize its proxy.

12 Conclusion

12.1 Overall assessment

2factor is a carefully built protocol. The parts that usually break in tranche-and-vault systems — share arithmetic, fee accounting, collateral reconciliation, reentrancy ordering — are correct here, and correct in a way that survives being derived symbolically rather than merely tested. The prior external review's findings are properly fixed. The test suite is large, honest, and covers the paths it claims to.

What this review found instead is a system whose *availability* is more fragile than its *solvency*. Three separate mechanisms can stop a holder from leaving: the fee curve reaching 100% and being converted into a revert (MED-01), a single price feed going quiet (MED-03), and the keeper's pause (§10). None of them loses money directly. All of them would be experienced by a user as losing money, because the position they cannot exit keeps moving.

12.2 Confirmed severe risks

None at Critical or High. Four Medium findings, all confirmed by executable proof-of-concept: the redemption freeze (MED-01), the missing first-depositor protection on the notes (MED-02), the single-oracle dependency (MED-03), and the unbounded AUM fee (MED-04).

12.3 Unresolved issues

All twenty-five findings are open at the reviewed commit. No maintainer response or fix was supplied during this engagement, so nothing has been retested.

12.4 Assumptions this report rests on

- The underlying collateral token is non-rebasing, non-fee-on-transfer, and has no transfer hooks. The code states this in NatSpec and enforces it nowhere.
- Chainlink feeds, Curve pools, Uniswap v4, CoW Protocol and Morpho Blue behave as their interfaces and documentation describe. None was tested against a live deployment.
- The owner and keeper keys are held by parties acting in good faith, since nothing in code constrains them.
- The fee curve, DR target, rebalance cadence and vault band parameters actually deployed are the ones in `init()` or something close to them. Several findings' severity moves if they are not.

12.5 Recommended actions before deployment

In priority order.

1. **Decide, in writing, what the fee curve is allowed to do to a redemption** (MED-01). Either clamp the fee below 100%, or gate only mints. This is the finding most likely to become a public incident, because it will look like a freeze rather than a fee.
2. **Add a fallback price path or an unpriced emergency exit** (MED-03). A pro-rata in-kind redemption needs no oracle to be safe.
3. **Bound the AUM fee and make it time-weighted** (MED-04), and put a timelock in front of the owner.

4. **Add first-depositor protection to the notes** (MED-02), and seed both notes in the same transaction that deploys them.
5. **Clip `vmDR`** (LOW-01) — one line, and it removes a whole class of panic revert.
6. **Restore the fee-policy ordering guard** (LOW-04). The failure it prevents is unrecoverable without an upgrade.
7. **Fix the vault redemption asymmetry** (LOW-06) and add the mirror of the existing deposit-side conservation test.
8. **Add events to every privileged setter** (INFO-09). Cheapest meaningful security improvement in the repository.
9. **Run the disabled fuzz suites at scale in CI**, and add a stateful multi-actor invariant campaign over the rollover and rebalance paths. That is the gap this engagement did not close.
10. **Run the fork suites** against real Curve, Uniswap v4, Morpho and CoW deployments before the vault layer holds user money.

12.6 Residual risk

After the ten actions above, the residual risk is concentrated in three places that no amount of contract review removes: the correctness of a single price feed per asset, the trustworthiness of the owner key, and the economic assumption that junior demand and senior demand stay close enough to the target ratio that the system does not spend long in the states where its own policy curves become punitive. The first two are addressable operationally. The third is the product, and it should be simulated before it is capitalised.

A File inventory and review status

All 58 in-scope files. *Full* means every line was read in context and its call paths traced; *Full (interface)* means the declaration was read and checked against its implementations and against the upstream protocol it mirrors. No in-scope file was left unreviewed.

File	Lines	Kind	Status
core-contracts/contracts			
PerpController.sol	1416	contract	Full
Router.sol	471	contract	Full
bonds/BondsManager.sol	772	contract	Full
bonds/Tranche.sol	118	contract	Full
perps/PerpTranche.sol	321	contract	Full
perps/PerpSr.sol	31	contract	Full
perps/PerpJr.sol	70	contract	Full
strategies/PerpBalancer.sol	641	contract	Full
strategies/FeePolicy.sol	421	contract	Full
strategies/BondIssuer.sol	211	contract	Full
oracles/CachedOracle.sol	62	contract	Full
oracles/CLOracleWrapper.sol	64	contract	Full
oracles/L2CLOracleWrapper.sol	63	contract	Full
_utils/PerpHelpers.sol	252	library	Full
_utils/LineHelpers.sol	127	library	Full
_utils/TrancheHelpers.sol	76	library	Full
_utils/MathHelpers.sol	72	library	Full
_utils/PerpBalancerHelpers.sol	47	library	Full
_utils/Shrink.sol	26	library	Full
_utils/ERC20Helpers.sol	15	library	Full
_interfaces/IBondsManager.sol	320	interface	Full (interface)
_interfaces/IPerpController.sol	220	interface	Full (interface)
_interfaces/IPerp.sol	116	interface	Full (interface)
_interfaces/Errors.sol	104	interface	Full (interface)
_interfaces/ITranche.sol	81	interface	Full (interface)
_interfaces/IFeePolicy.sol	66	interface	Full (interface)
_interfaces/Types.sol	58	interface	Full (interface)
_interfaces/IPerpBalancer.sol	50	interface	Full (interface)
_interfaces/IBondIssuer.sol	38	interface	Full (interface)
_interfaces/IChainlinkOracle.sol	25	interface	Full (interface)
_interfaces/Constants.sol	17	interface	Full (interface)
_interfaces/IPriceOracle.sol	15	interface	Full (interface)
_interfaces/IERC20Burnable.sol	9	interface	Full (interface)
_interfaces/IOwnable.sol	8	interface	Full (interface)
_external/ProxyImports.sol	8	interface	Full (interface)
vaults/contracts			
UsdPerpSrVault.sol	1503	contract	Full
UnderlyingPerpVault.sol	599	contract	Full

continued overleaf

File	Lines	Kind	Status
routers/UsdPerpSrVaultCowBroker.sol	445	contract	Full
routers/UsdPerpSrVaultRouter.sol	190	contract	Full
routers/UnderlyingPerpVaultRouter.sol	165	contract	Full
routers/PerpVaultRouter.sol	113	contract	Full
strategies/UniswapV4CbBtcUsdcStrategy.sol	305	contract	Full
strategies/CurveCbBtcUsdcStrategy.sol	290	contract	Full
strategies/BaseDexStrategy.sol	80	contract	Full
_utils/PerpVaultHelpers.sol	338	library	Full
_utils/RouterHelpers.sol	46	library	Full
_interfaces/IDexStrategy.sol	86	interface	Full (interface)
_interfaces/GPv2Order.sol	85	interface	Full
_interfaces/IConditionalOrder.sol	69	interface	Full (interface)
_interfaces/ICurveStableSwapNG.sol	52	interface	Full (interface)
_interfaces/CommonErrors.sol	51	interface	Full (interface)
_interfaces/IComposableCoW.sol	47	interface	Full (interface)
_interfaces/ICurveTwoCryptoNG.sol	45	interface	Full (interface)
_interfaces/IMorpho.sol	23	interface	Full (interface)
_interfaces/Types.sol	18	interface	Full (interface)
_interfaces/ICowFlashLoanRouter.sol	16	interface	Full (interface)
_interfaces/IGPv2Settlement.sol	16	interface	Full (interface)
_external/ProxyImports.sol	8	interface	Full (interface)

`GPv2Order.sol` is marked *Full* rather than *Full (interface)* because it contains executable assembly: its `hash` function keccaks 416 bytes starting one word before the struct pointer, after temporarily overwriting that word with the type hash and restoring it. That encoding was checked field by field against the canonical CoW implementation — twelve value-type fields, one word each, in EIP-712 declaration order, with `kind` and the two balance markers carried pre-hashed. It matches, which is what makes the broker's ERC-1271 digests agree with the settlement's.

B Privileged function inventory

Every function in scope carrying an authorisation modifier. Functions with no modifier are permissionless; the notable permissionless state-mutating ones are listed at the end.

Function	Visibility	Gate
PerpController		
`init`	external	`initializer`
`updateIssuer`	external	`onlyOwner`
`updateFeePolicy`	external	`onlyOwner`
`updatePerpBalancer`	external	`onlyOwner`
`updateRolloverEligibility`	external	`onlyOwner`
`updateDRTargets`	external	`onlyOwner`
`updateMinCleanupVal`	external	`onlyOwner`
`updateLiquidityThresholds`	external	`onlyOwner`
`updateRebalanceWindow`	external	`onlyOwner`
`setUnderlyingOracle`	external	`onlyOwner`
`resetUnderlyingOracle`	external	`onlyOwner`
`updateKeeper`	external	`onlyOwner`
`pauseSystemOps`	external	`onlyKeeper`
`pauseUserOps`	external	`onlyKeeper`
`updateValueCap`	external	`onlyKeeper`
`pullProtocolFees`	external	`onlyProtocolFeeCollector`
BondsManager		
`init`	external	`initializer`
`addIssuer` / `removeIssuer`	external	`onlyOwner`
`addAsset` / `removeAsset`	external	`onlyOwner`
`updateOracle`	external	`onlyOwner`
`createBond`	external	`onlyIssuer`
Tranche		
`init`	public	`initializer`
`mint` / `burn`	external	`onlyOwner` (the BondsManager)
PerpTranche / PerpSr / PerpJr		
`init`	public	`initializer`
`updateController`	external	`onlyOwner`
`mintTo` / `burnFrom`	external	`onlyController`
`transferReservesTo`	public	`onlyController`
`handleTransferReservesIn`	public	`onlyController`
`registerTranche`	external	`onlyController`
`logBalance`	public	`onlyController`
`recover`	external	`onlyController`
`primeApproval` / `revokeApproval`	external	`onlyController`
`bondDepositsForRollover`	external	`onlyController`

continued overleaf

Function	Visibility	Gate
FeePolicy		
init	external	initializer
updateFeeParams	public	onlyOwner
updateRebalanceParams	public	onlyOwner
updateProtocolFeeConfig	external	onlyOwner
updateAumFeePerc	external	onlyOwner
BondIssuer		
init	external	initializer
updateBondConfig	external	onlyOwner
updateIssuanceTimingConfig	public	onlyOwner
UnderlyingPerpVault		
init	public	initializer
updateKeeper	public	onlyOwner
updateTargetDR	public	onlyOwner
updateMinRebalanceAmt	public	onlyOwner
updateRebalanceTimingParams	public	onlyOwner
updateMaxFlashFeePerc	public	onlyOwner
initDeposit	external	onlyOwner
pause / unpause	external	onlyKeeper
UsdPerpSrVault		
init	external	initializer
updateKeeper	public	onlyOwner
updateSwapper	public	onlyOwner
updateTargetDR	public	onlyOwner
updateOracleDeviationPerc	public	onlyOwner
updateDexStrategy	public	onlyOwner
updateOpLimits	public	onlyOwner
updateSessionParams	public	onlyOwner
updateHealthCheckParams	public	onlyOwner
initDeposit	external	onlyOwner
pause / unpause	external	onlyKeeper
swap	external	onlySwapper
UsdPerpSrVaultCowBroker		
flashLoanAndCallBack	external	onlyCowRouter, onlyPinnedLender, whenNoLoanInFlight
onMorphoFlashLoan	external	onlyLender
provideInventory	external	onlyTrampoline, whenLoanInFlight

Permissionless state-mutating entry points. PerpController: mint2, redeem2, flashMint2, flashRedeem2, rebalance, rollover, upkeep. BondsManager: deposit, redeem, redeemAll, redeemMature, redeemImmature, mature, tryMature. BondIssuer: issue, cleanupMature. Vaults:

`deposit`, `redeem`, `rebalance`, `upkeep`. Broker: `settleToVault`. Oracles: `cachePrice`, `resetCache`. Routers and strategies: everything.

C Tool versions and commands

Every command below was executed in an isolated copy of the repository. The reviewed working tree was never modified.

Environment setup

```
# isolated copy; node_modules shared by symlink, nothing written to the original
rsync -a --exclude node_modules --exclude .git --exclude artifacts \
      --exclude cache --exclude coverage --exclude typechain-types \
      <repo>/ <work>/
ln -s <repo>/node_modules                  <work>/node_modules
ln -s <repo>/core-contracts/node_modules   <work>/core-contracts/node_modules
ln -s <repo>/vaults/node_modules           <work>/vaults/node_modules

nvm use 22.15.0
```

Build

```
cd <work>/core-contracts && npx hardhat compile   # 94 files, cancun
cd <work>/vaults         && npx hardhat compile   # 136 files, cancun
```

Existing suites

```
cd <work>/core-contracts && FUZZ_RUNS=0 PATH_FUZZ_RUNS=0 npx hardhat test
cd <work>/vaults         && npx hardhat test test/*.ts
```

Proof-of-concept suites

```
# copy the delivered proofs into the packages' test trees first
cp -r proofs-of-concept/core   <work>/core-contracts/test/audit
cp -r proofs-of-concept/vaults <work>/vaults/test/audit

cd <work>/core-contracts
FUZZ_RUNS=0 PATH_FUZZ_RUNS=0 npx hardhat test test/audit/*.ts
cd <work>/vaults
npx hardhat test test/audit/*.ts
```

Coverage

```
cd <work>/core-contracts && FUZZ_RUNS=0 PATH_FUZZ_RUNS=0 npx hardhat coverage
cd <work>/vaults         && npx hardhat coverage --testfiles 'test/*.ts'
```

Static analysis

```
cd <work>/core-contracts && npx solhint "contracts/**/*.sol"
```

```
cd <work>/vaults         && npx solhint "contracts/**/*.sol"

# Slither runs ‘hardhat clean --global‘, so give it its own copy
python3.12 -m venv .venv-slither
.venv-slither/bin/pip install cbor2 --only-binary :all:
.venv-slither/bin/pip install slither-analyzer          # 0.11.6
cd <slither-work>/core-contracts && slither . --exclude-dependencies \
    --json slither-core.json
cd <slither-work>/vaults         && slither . --exclude-dependencies \
    --json slither-vaults.json
```

Report

```
cd report && tectonic -X compile report.tex     # produces report.pdf
```

Versions

Node.js	v22.15.0
npm	11.18.0
yarn	4.9.1
Hardhat	2.29.0
solc	0.8.35+commit.47b9dedd
ethers	6.16.0
mocha	11.7.6
chai	4.5.0
solidity-coverage	0.8.17
solhint	6.2.3
Slither	0.11.6
typechain	8.3.2
@openzeppelin/contracts	5.6.1
@openzeppelin/contracts-upgradeable	5.6.1
@openzeppelin/hardhat-upgrades	3.9.1
@uniswap/v4-core	1.0.2
tectonic (LaTeX)	0.15.0
Host	macOS 15.7.7, x86_64

D Candidate-finding disposition

Thirty-one hypotheses were opened during the review: 16 became findings and 15 were invalidated. Both sets are listed. The invalidated ones carry the reason, so the same ground does not have to be re-walked in a future engagement. Findings INFO-02 through INFO-10 are absent from this table because they were observations from the line-by-line pass rather than hypotheses under test.

Hypothesis	State	Reason
Fee accounting can drift so `protocolFeeBals` exceeds the balance	Confirmed	LOW-04
Donation to a note inflates price-per-share	Confirmed	MED-02
Fee curve can block redemption	Confirmed	MED-01
Oracle outage blocks exits	Confirmed	MED-03
AUM fee is unbounded	Confirmed	MED-04
`vmDR` is unbounded into the fee curve	Confirmed	LOW-01
Sub-unit bond deposits mint nothing	Confirmed	LOW-02
`redeemImmature` rounding breaks pairing	Confirmed	LOW-03
`upkeep` `NoOp` breaks router flags	Confirmed	LOW-05
Vault resting balance is asymmetric	Confirmed	LOW-06
Zero lower band disables the vault	Confirmed	LOW-07
Router arbitrary approvals	Confirmed	LOW-08 (already documented in-source)
Settlement price is caller-chosen	Confirmed	LOW-09
Issuer misconfiguration halts minting	Confirmed	LOW-10
`valueRatio` can be zero at rollover	Confirmed	LOW-11
Fee settlement emits no sync event	Confirmed	INFO-01
Flash-loan sandwich of `rebalance()` extracts the transferred value	Invalidated	The sequence composes — the reentrancy guard is per call, not per transaction — but the round trip costs ≈1% on the mint plus ≈2.9% on the redeem plus two flash base fees against a capture of at most ≈0.83%. Unprofitable at the shipped curves. Recorded as an economic risk because the margin depends entirely on parameters.

continued overleaf

Hypothesis	State	Reason
Redemption can be made non-pro-rata by choosing which tranches to take	Invalidated	`computeRedemptionAmts` slices every reserve entry by the same ratio; there is no per-asset selection anywhere in the path.
Rollover leaks value from one note to the other	Invalidated	The plan sizes each deposit as V/valueRatio and the senior tokens minted are worth exactly V back, because trancheSupply/totalClaim = openPrice × trancheRatio is maintained by deposit and proportional redemption. Ceiling rounding costs PerpJr $O(1)$ wei per tranche, in PerpSr's favour.
The CoW solver can make the vault trade below its floor	Invalidated	The pre-hook's arguments are solver-controlled, but `vault.swap` re-checks price, direction, DR gap, session budget and minimum size on-chain against the oracle; and the broker's own solvency (it must repay the flash loan from what the settlement leaves it) bounds the quantities independently. Both order directions were traced.
The CoW broker's ERC-1271 signature can be replayed	Invalidated	GPv2 records `filledAmount` per order UID and rejects a second fill of a fill-or-kill order; and every fill re-prices against the live oracle at settlement time, so a stale signature is not a stale price.
An unregistered tranche can be injected into a note's reserve set	Invalidated	`_syncTranche` runs only for tranches involved in a controller-driven transfer, and `registerTranche` only ever adds the current bond's legs. The reserve set cannot be grown by an outsider — which also means `ReserveOverflow` cannot be weaponised. (The converse, that such a tranche is then stranded, is INFO-02.)
`cleanupMature`'s reverse iteration with swap-and-pop can skip or re-read an entry	Invalidated	Removal moves the element at index $L-1$ into the removed slot, and the loop invariant $L \geq i$ holds throughout, so the moved element has always already been processed and `at(i-1)` is always in range.
A note can be left holding a worthless tranche it can never prune	Invalidated	`redeemMature` burns the full requested amount even when the claim is zero, so the balance reaches zero and `_syncTranche` removes the entry.
Bond collateral can be under-paid to the last redeemer by accumulated flooring	Invalidated	Both redemption paths give the final redeemer claim × supply/supply, which is the exact remaining claim, not a floored fraction.
`withCachedPrice` can be desynchronised by a third party calling `resetCache` mid-operation	Invalidated	The cache is transient and per-transaction; a reset only forces a re-fetch, which returns the same per-block value. Recorded as INFO-04 for surface reduction, not as a live issue.

continued overleaf

Hypothesis	State	Reason
Implementation contracts can be initialised and used to hijack a proxy	Invalidated	Every upgradeable implementation calls `_disableInitializers()` in its constructor.
`PerpTranche`'s ERC-7201 slot collides with inherited OpenZeppelin storage	Invalidated	The hardcoded slot matches `erc7201Slot("2ff.storage.PerpTranche")`, which the repository's own test pins, and namespaced slots cannot collide with sequential ones.
A hostile underlying token's transfer hook can reenter a mutating path	Invalidated	Every controller and bonds-manager entry point is `nonReentrant`; the notes are `onlyController`. A reentrant call from a fee payout or a bond deposit reaches nothing mutable. Separately, hooked tokens are excluded by the stated collateral assumption.
`Router.redeem3` can be made to pay a caller more than they redeemed	Invalidated	It pays out the router's whole balance, but the router's invariant is that it holds nothing between transactions; within one call the balance is exactly what the redemption produced. The residual risk is the stranded-balance case, LOW-08.
The `maxTrancheCount` ceiling can be reached to brick issuance	Invalidated	The set size is bounded by maxMaturitySec/minIssueTimeIntervalSec and cannot be grown by an outsider (see above). Reported instead as a configuration constraint in LOW-10.

E Prior audit verification

PeckShield's March 2026 report (2026-021) raised six findings at commit `dc1e08e`. Each was checked against the reviewed commit.

ID	Title	Status	Verification at `2e527da`
PVE-001	`bondCDR` division by zero	Fixed	`TrancheHelpers.bondCDR` now guards both branches: `valueRatio <= 0` returns 0 for senior, `valueRatio >= ONE` returns `INF_BOND_CDR` for junior.
PVE-002	`incrementTrancheBal` value not accumulated	Fixed	Now `rtd.balance += trAmt.amount` followed by `rtd.value = rtd.data.value(rtd.balance)` on the memory reference, so both fields track.
PVE-003	`redeemImmature` returned non-zero amounts on a zero return	Fixed	Early return is now `(0, TokenAmount(bd.senior, 0), TokenAmount(bd.junior, 0))`.
PVE-004	`_cloneReserveTrancheData` out-of-bounds	Fixed	Loop bound is now `Math.min(rtd.length, capacity)`.
PVE-005	`value()` wrong on zero bond claim	Fixed	Now returns 0 when `trancheSupply <= 0 || totalClaim <= 0`, and the `underlyingOpenPrice` branch is gone entirely.
PVE-006	Trust in admin keys	Mitigated	Unchanged in code. This review re-raises it as §10 and MED-04, because the capability set has grown since March — the vault layer, its keeper, and the AUM fee path are all new.

Note that LOW-03 in this report is adjacent to PVE-003 but distinct: PeckShield's finding was about the *return values* on a zero-underlying redemption, which is fixed; this one is about the junior burn amount rounding to zero while the senior burn and the underlying payout do not.

F Glossary

Bond
A fixed-maturity container holding underlying collateral, against which one senior and one junior tranche token are minted at a fixed ratio, with the oracle price at creation recorded as `openPrice`.

Tranche
An ERC-20 claim on part of a bond. Senior has priority; junior takes the residual. Deployed as EIP-1167 clones by the BondsManager.

`trancheRatio`
The tranche's share of the bond at creation (1e18 = 100%). Fixed.

`valueRatio`
The tranche's current share of the bond's claim (1e18 = 100%). Moves with the price.

`bondCDR`
Collateralisation/deviation ratio of the parent bond; `trancheRatio / valueRatio` for senior, the complement for junior. 1e18 means fairly priced.

Perp note
`PerpSr` or `PerpJr`: an ERC-20 claim on a rolling basket of tranches of one seniority, plus underlying.

`subDR`
Subscription deviation ratio. (PerpJr TVL/PerpSr TVL)/targetSystemRatio. 1e18 means the two legs sit at their target proportion.

`vmDR`
Volatility-multiplier deviation ratio. PerpJr blended VM/targetPerpJrVM. 1e18 means the junior leg carries its target leverage.

Volatility multiplier (VM)
A tranche's price sensitivity relative to the underlying; `ONE / valueRatio` for a junior tranche, so it rises as the junior's claim is consumed.

Rollover
Cycling stale senior tranches out of PerpSr, through PerpJr, into freshly minted ones — designed to be value-neutral to both notes.

Rebalance
The scheduled transfer of reserve value between the two notes to hold `targetSystemRatio`.

Flash mint / flash redeem
Minting a note directly from underlying, or redeeming it directly to underlying, rather than through tranches. The redeem side draws on PerpJr's idle liquidity.

Session (vaults)
A per-window budget and locked trade direction, sized at `maxDeploymentVelocityPerc` of vault TVL.

Equilibrium band
The DR interval a vault reads as balanced and declines to trade inside.

Resting underlying
Underlying a vault holds between trades, left over when a flash redemption's realised fee is lower than the worst case it sized for.

CoW broker
A whitelisted counterparty that fills the vault's passive swap leg over CoW Protocol, funding itself with a Morpho flash loan and signing orders by ERC-1271.

G Disclaimer

This report reflects a point-in-time review of the source code at the commit identified on the cover, performed under the constraints and limitations recorded in §3.6. It is not a guarantee, warranty, or certification that the software is free of defects, and it is not investment, legal or tax advice.

Security review reduces risk; it does not remove it. The findings here are those that one reviewer identified in the time recorded. Defects may exist that were not found, in code that was reviewed as well as in code that was out of scope. A clean test run is not evidence of security. High coverage is not evidence of adversarial coverage. The absence of a Critical or High finding in this report is a statement about this review, not about the software.

This review does not cover deployment, configuration, key management, off-chain infrastructure, governance process, front ends, or economic parameters chosen after the reviewed commit — several of which materially affect the severity of findings recorded here.

Model-assisted review carries its own failure modes, described in §1.4. Readers should weight claims by the confidence rating attached to each finding, and should treat the proof-of-concept tests delivered with this report — rather than its prose — as the primary evidence.

The recipient is encouraged to commission independent review from additional parties and to operate a public bug bounty before and after deployment.

End of report — 2factor Smart Contract Security Assessment, version 1.0